

СОДЕРЖАНИЕ

Введение	3
Глава 1. Теоретические основы защиты медиапотокa.....	4
1.1. Принципы конфиденциальности, целостности и доступности данных	4
1.2. Криптографические основы защиты информации	5
1.3. Специфика защиты медиапотока в реальном времени.....	5
Глава 2. Анализ угроз потоковой передачи медиaданных	7
2.1. Модель участников и границы доверия.....	7
2.2. Защищаемые активы и предположения модели угроз	8
2.3. Модель нарушителя	9
2.4. Временные ограничения и сетевые факторы в медиасистемах	10
2.5. Классификация векторов атак и проблема верификации происхождения данных	11
2.6. Угрозы сетевого уровня и механизмы атак повторного воспроизведения	11
2.7. Угрозы нарушения порядка, задержки и устаревания данных	12
2.8. Дифференциация атак представления и инъекционных атак	13
2.9. Использование генеративного искусственного интеллекта и технологий глубокой подделки в инъекционных векторах	13
2.10. Последствия компрометации сервера-маршрутизатора	14
2.11. Переход к парадигме аппаратного подтверждения происхождения данных	15
2.12. Выводы по результатам анализа угроз	16
Глава 3. Анализ существующих подходов и обоснование выбранной архитектуры.....	16
3.1. Стандартизированные подходы к обеспечению подлинности медиaданных.....	17
3.1.1. Индустриальные инициативы по обеспечению подлинности медиaданных.....	17
3.1.2. Архитектура и ограничения стандарта C2PA.....	17
3.1.3. Проблема задержек и сегментации в C2PA	18
3.2. Транспортный уровень защиты медиапотока	19
3.2.1. Транспортные протоколы: SRT, WebRTC и QUIC.....	19
3.3. Криптографические примитивы.....	21
3.3.1. Хеш-функции как механизм обеспечения целостности	21
3.3.2. Сравнительный анализ SHA-256 и SHA-512.....	21
3.3.3. Необходимость асимметричной криптографии	22
3.3.4. Алгоритм RSA и его ограничения	22

3.3.5. ECDSA и проблема случайных чисел.....	23
3.3.6. Ed25519 как базовое решение	23
3.4. Аппаратный корень доверия и использование доверенного платформенного модуля	24
3.4.1. Понятие аппаратного корня доверия	24
3.4.2. Назначение доверенного платформенного модуля.....	24
3.4.3. Обоснование использования доверенного платформенного модуля для подтверждения происхождения медиапотока	24
3.4.4. Использование доверенного платформенного модуля на этапе рукопожатия	25
3.4.5. Разделение ролей долговременного и временного ключей.....	25
3.5. Формализация требований к протоколу	26
3.5.1. Аппаратный корень доверия	26
3.5.2. Устойчивость к активным манипуляциям и атакам повтора.....	26
3.5.3. Минимальные накладные расходы для режима реального времени...	27
3.5.4. Масштабируемость сервера-маршрутизатора	27
3.5.5. Сквозная проверка на стороне консьюмера.....	27
3.6. Итог главы.....	28
Глава 4. Программная реализация и тестирование разработанного решения	28
4.1. Сценарий работы разработанного приложения.....	28
4.2. Демонстрация установления доверенной сессии.....	30
4.3. Демонстрация передачи и проверки кадров	32
4.4. Методика и сценарии модульного тестирования	35
4.4.1. Тесты установления доверенной сессии	36
4.4.2. Криптографические тесты целостности данных	38
4.4.3. Сетевые-протокольные тесты актуальности и порядка передачи	39
4.4.4. Сессионные тесты принадлежности данных	40
4.4.5. Тесты пороговой политики реакции на нарушения	41
4.5. Выводы по результатам тестирования.....	42
Глава 5. Анализ результатов.....	43
5.1. Вычислительная нагрузка	43
5.2. Сетевые накладные расходы.....	44
5.3. Применимость решения.....	44
5.4. Анализ результатов.....	45
Заключение	47
Список использованных источников.....	48

ВВЕДЕНИЕ

Современные системы передачи медиаданных требуют подтверждения подлинности изображения и звука. Это важно для видеоконференций, удаленной идентификации, телемедицины и систем контроля доступа. Развитие генеративного искусственного интеллекта (ИИ) и средств подмены видеопотока повышает риск внедрения синтетических данных.

Целью работы является разработка протокола криптографической аутентификации медиапотока, обеспечивающего проверку происхождения, целостности и актуальности видеокадров при передаче через сервер-маршрутизатор.

Сценарий включает три роли: продюсер формирует видеопоток, сервер-маршрутизатор передает его получателям, а консюмер проверяет подлинность кадров. Сервер и сеть не считаются полностью доверенными. Протокол защищает поток от атаки Man-in-the-Middle (MITM, атака посредника), подмены кадров, повторной отправки старых кадров и внедрения синтетического контента.

Объектом исследования являются системы потоковой передачи медиаданных в реальном времени.

Предметом исследования являются методы криптографического подтверждения происхождения и целостности видеокадров.

Для достижения цели необходимо:

1. Рассмотреть особенности потоковой передачи медиаданных.
2. Проанализировать угрозы подмены, повтора и синтетической генерации.
3. Исследовать криптографические методы защиты медиапотока.
4. Определить ограничения существующих подходов.
5. Сформулировать требования к протоколу.
6. Разработать структуру криптографической метки и алгоритм проверки.
7. Предложить архитектуру решения.

В работе рассматриваются SHA (Secure Hash Algorithm, безопасный алгоритм хеширования), цифровая подпись Ed25519, сессионные идентификаторы, временные метки и одноразовые случайные значения.

Практическая значимость работы заключается в применении подхода в системах доверенной потоковой передачи видео.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ЗАЩИТЫ МЕДИАПОТОКОВ

1.1. Принципы конфиденциальности, целостности и доступности данных

В распределённых цифровых системах защита информации является базовым условием надёжной работы сервисов и сохранности данных. Рост объёмов передаваемой и обрабатываемой информации повышает требования к мерам безопасности и усиливает последствия инцидентов, включая экономический ущерб и репутационные потери.

В качестве теоретической основы информационной безопасности используется триада CIA (Confidentiality, Integrity, Availability), включающая конфиденциальность, целостность и доступность. Указанная модель применяется при анализе и проектировании защищённых информационных систем и закреплена в стандартах.

Конфиденциальность означает недоступность информации для лиц без соответствующих полномочий и предполагает предоставление доступа только авторизованным пользователям в пределах их прав. В правовом смысле конфиденциальность рассматривается как режим ограничения распространения информации; в соответствии с Федеральным законом Российской Федерации № 149-ФЗ «Об информации, информационных технологиях и о защите информации» лицо, получившее доступ к информации, обязано не передавать её третьим лицам без согласия правообладателя, если иное не предусмотрено законодательством. На практике конфиденциальность обеспечивается сочетанием технических и организационных мер, включая шифрование при хранении и передаче, управление доступом и аутентификацию, регламенты и контроль доступа к инфраструктуре. К типичным угрозам относятся несанкционированный доступ (включая социальную инженерию), утечки из-за ошибок или злоупотреблений персонала, перехват при передаче и компрометация учётных записей.

Целостность характеризует сохранение точности и непротиворечивости данных и гарантирует отсутствие несанкционированных изменений. Различают физическую целостность, связанную с защитой от потерь вследствие отказов оборудования и аварий, и логическую целостность, относящуюся к корректности данных на уровне программных и информационных структур. Целостность поддерживается криптографическими средствами (хеш-функции и электронные подписи), резервным копированием, контролем версий и мониторингом, а также

организационными процедурами разграничения полномочий и контроля изменений. Основные угрозы включают преднамеренную модификацию данных, ошибки пользователей, технические сбои и вредоносное ПО, а также прикладные атаки, направленные на эксплуатацию уязвимостей.

Доступность означает возможность своевременного доступа авторизованных пользователей к данным и сервисам. Она должна обеспечиваться без неоправданного снижения удобства и работоспособности системы из-за чрезмерных ограничений или перегрузки механизмов аутентификации. Нарушение доступности вызывается отказами оборудования, программными сбоями, перегрузкой ресурсов и атаками типа отказа в обслуживании. Для снижения рисков применяются резервирование, отказоустойчивые архитектуры, резервное копирование, мониторинг и планы восстановления после сбоев.

1.2. Криптографические основы защиты информации

Криптографические методы обеспечивают конфиденциальность, целостность и аутентичность данных. Симметричное шифрование использует общий ключ и отличается высокой производительностью, но требует безопасного распределения ключа. Асимметричное шифрование опирается на пару ключей (открытый и закрытый), упрощает распределение ключей и используется в задачах аутентификации и обмена секретами, но обычно уступает симметричным схемам по скорости. Обмен ключами применяется для согласования общего секрета между сторонами без раскрытия ключевого материала третьей стороне. Цифровая подпись обеспечивает проверяемую подлинность отправителя и обнаружение изменений данных за счёт формирования подписи закрытым ключом и проверки открытым ключом.

1.3. Специфика защиты медиапотока в реальном времени

Классические свойства информационной безопасности приобретают особую интерпретацию при передаче медиапотока в реальном времени. Для обычных файловых данных целостность чаще всего означает точное восстановление всего переданного объекта, а доступность — возможность получить данные без потери их содержательной полноты. Для медиапотока такой подход является недостаточным, поскольку видеокадр имеет ценность только в пределах ограниченного временного интервала. Кадр, доставленный слишком поздно, может быть формально коррект-

ным, но практически непригодным для воспроизведения и проверки текущего состояния источника.

В условиях потоковой передачи видео основное значение приобретают три свойства: целостность кадра, подтверждение происхождения кадра и актуальность кадра. Целостность означает, что полезная нагрузка и связанные с ней служебные поля не были изменены после формирования подписи. Подтверждение происхождения означает, что кадр был сформирован доверенным источником, а не внедрён промежуточным узлом, сервером или вредоносным программным компонентом. Актуальность означает, что кадр относится к текущей сессии передачи и не является повторно отправленным фрагментом старого потока.

Для реализации указанных свойств одной защиты транспортного канала недостаточно. Шифрование соединения позволяет снизить риск перехвата и модификации данных при передаче, однако не решает задачу сквозного подтверждения происхождения медиаданных. Если сервер-маршрутизатор или программная среда источника будут скомпрометированы, защищённый транспортный канал сам по себе не докажет, что полученный видеокادر действительно был сформирован доверенной камерой. Поэтому в рассматриваемой задаче требуется дополнительный прикладной уровень проверки, независимый от доверия к серверу и сетевой инфраструктуре.

В рамках разрабатываемого подхода каждый кадр рассматривается как самостоятельный проверяемый объект. Криптографическая проверка выполняется не для всего видеопотока целиком, а для отдельной полезной нагрузки вместе со служебными полями: идентификатором сессии, порядковым номером и временной меткой. Такое построение позволяет обнаруживать изменение содержимого кадра, повторную отправку старых данных, нарушение порядка кадров и принадлежность кадра к другой сессии. При этом протокол не зависит от внутренней структуры видеокодека, поскольку проверяемой единицей выступает ограниченный по размеру полезной нагрузки.

Модель формирования и проверки подписи кадра представлена на рисунке 1.1. В подписываемые данные включаются идентификатор сессии, порядковый номер кадра, временная метка и полезная нагрузка. Полученная подпись помещается в структуру кадра и используется консюмером для проверки целостности и подлинности данных.

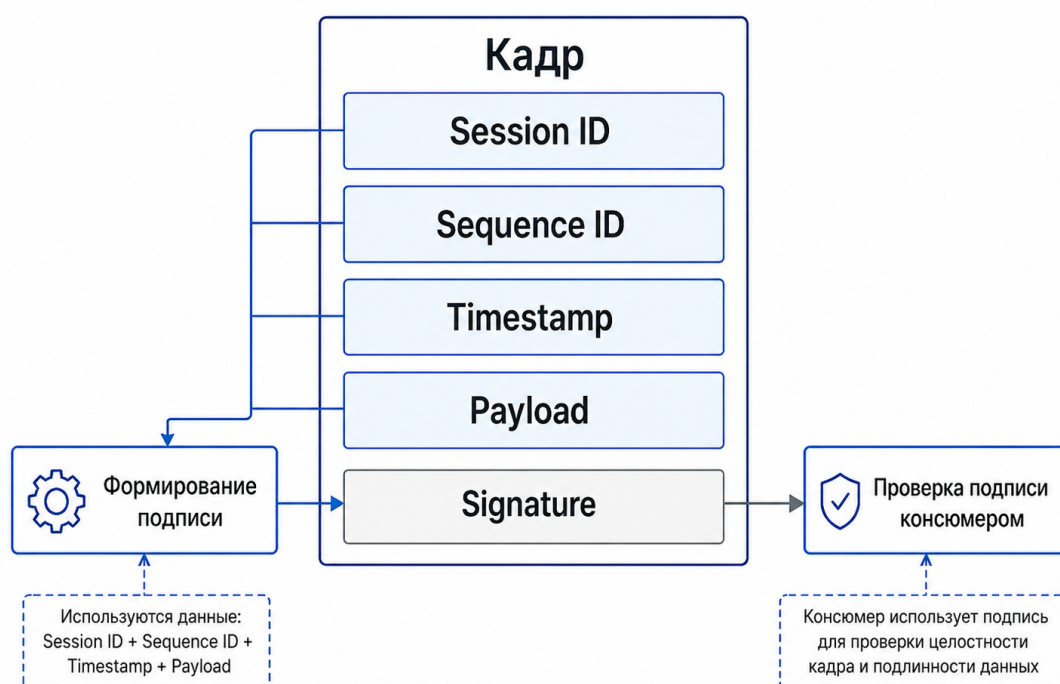


Рис.1.1. Модель формирования и проверки подписи кадра медиапотока

Следовательно, защита медиапотока в реальном времени должна сочетать криптографическую целостность, проверку источника, контроль актуальности и учёт временных ограничений передачи. Именно эти особенности отличают задачу от классической защиты файловых данных и обосновывают необходимость специализированного протокола непрерывной криптографической аутентификации медиапотока.

ГЛАВА 2. АНАЛИЗ УГРОЗ ПОТОКОВОЙ ПЕРЕДАЧИ МЕДИАДААННЫХ

2.1. Модель участников и границы доверия

В рассматриваемой системе выделяются три основные роли: продюсер, сервер-маршрутизатор и консьюмер. Продюсер формирует видеопоток и подписывает данные, сервер-маршрутизатор обеспечивает передачу медиаданных между участниками, а консьюмер выполняет проверку подлинности и актуальности полученных кадров.

Модель участников и границы доверия представлены на рисунке 2.1. На схеме продюсер обозначен как доверенный источник, поскольку он формирует кадры, подписывает их и использует закрытый ключ, размещённый в доверенном платформенном модуле. Сервер-маршрутизатор выделен как недоверенный или частично доверенный промежуточный узел: он принимает поток, выполняет маршрутизацию по идентификатору сессии и может проверять сертификат, однако не выполняет проверку подписи каждого кадра. Консьюмер, в свою очередь, является стороной доверенной проверки, так как именно он проверяет сертификат, результат рукопожатия, подписи кадров и отбрасывает невалидные данные.

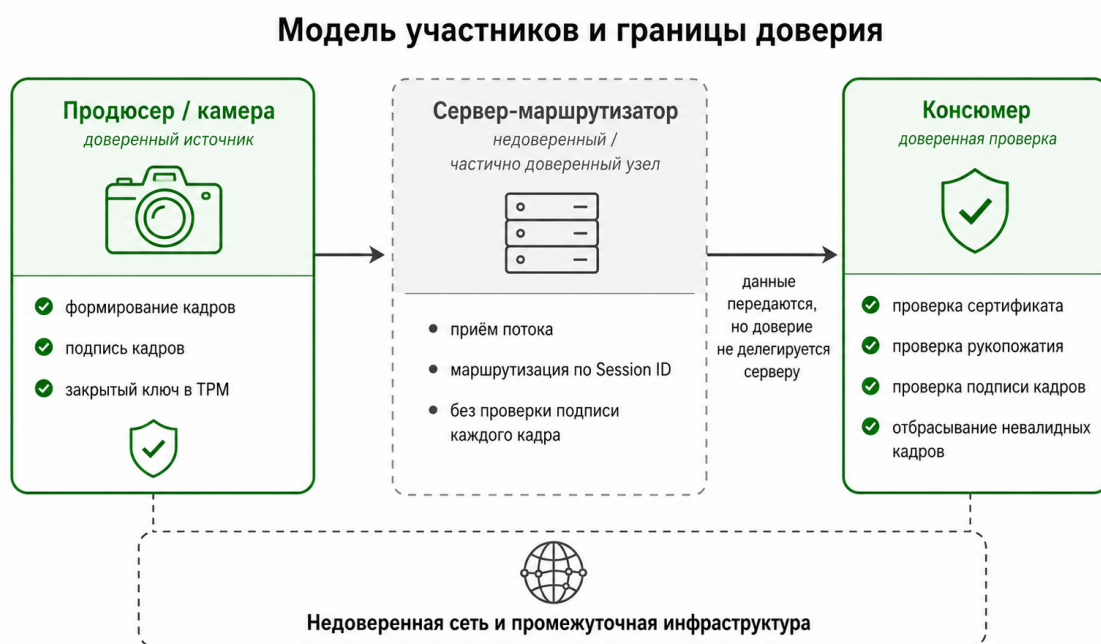


Рис.2.1. Модель участников и границы доверия

Сервер-маршрутизатор и сеть передачи не считаются полностью доверенными. Это означает, что промежуточный узел может быть скомпрометирован, ошибочно настроен или использован злоумышленником для задержки, удаления, перестановки либо подмены кадров. Поэтому данные могут передаваться через сервер, но доверие не делегируется серверу: окончательная проверка доверия к кадру должна выполняться на стороне консьюмера.

2.2. Защищаемые активы и предположения модели угроз

В рамках рассматриваемой системы защищаемым объектом является не только сам видеопоток, но и совокупность служебных данных, определяющих его

происхождение, принадлежность к текущей сессии и актуальность. К основным защищаемым активам относятся полезная нагрузка медиапотока, идентификатор сессии, порядковый номер, временная метка, сертификат камеры, временный открытый ключ текущей сессии и результат рукопожатия.

Полезная нагрузка должна сохранять целостность после формирования подписи. Любое изменение содержимого полезной нагрузки при передаче должно приводить к ошибке проверки. Идентификатор сессии используется для отделения текущего медиасеанса от предыдущих и предотвращает перенос корректно подписанных данных между разными сессиями. Порядковый номер позволяет контролировать порядок поступления данных и выявлять повторную отправку уже обработанных фрагментов. Временная метка необходима для оценки актуальности данных и отбрасывания устаревших сообщений.

Отдельным защищаемым активом является долговременная идентичность камеры. Она подтверждается сертификатом X.509, связанным с открытым ключом камеры. При этом закрытый ключ камеры не должен быть доступен программной среде в открытом виде, поскольку его компрометация позволила бы злоумышленнику формировать ложные данные с корректной криптографической подписью. Поэтому в модели угроз предполагается, что долговременный закрытый ключ хранится внутри доверенного платформенного модуля и не покидает устройство.

В модели также предполагается, что сервер-маршрутизатор и сеть передачи могут быть недоверенными. Они могут корректно выполнять передачу данных, но не рассматриваются как окончательный источник доверия. Окончательное решение о подлинности, целостности и актуальности медиаданных должно приниматься консьюмером на основании сертификата камеры, результата рукопожатия и подписи полезной нагрузки.

2.3. Модель нарушителя

Нарушитель в рассматриваемой системе может обладать возможностью пассивного наблюдения за сетевым трафиком, активного вмешательства в передачу данных и воздействия на промежуточную инфраструктуру. Он может перехватывать медиапоток, задерживать отдельные сообщения, удалять их, изменять порядок следования, повторно отправлять ранее записанные данные или пытаться внедрить собственную полезную нагрузку.

На сетевом уровне нарушитель может действовать как посредник между продюсером, сервером и консьюмером. В этом случае он способен выполнять атаку Man-in-the-Middle (MITM, атака посредника), подменять отдельные фрагменты данных или нарушать последовательность их доставки. Однако при корректной работе криптографического уровня он не должен иметь возможности сформировать корректную подпись для изменённой полезной нагрузки, поскольку временный закрытый ключ текущей сессии ему недоступен.

На уровне источника данных нарушитель может использовать виртуальную камеру, программный эмулятор видеоустройства, заранее записанный видеопоток или синтетический поток, созданный средствами генеративного искусственного интеллекта. Такой нарушитель стремится не просто изменить передаваемые данные, а заменить сам источник происхождения медиапотока. Поэтому защита должна быть направлена не только на обнаружение изменения битов при передаче, но и на подтверждение того, что поток связан с доверенным аппаратным устройством.

На уровне серверной инфраструктуры нарушитель может получить частичный или полный контроль над сервером-маршрутизатором. В этом случае сервер может передавать консьюмеру устаревшие данные, менять порядок сообщений, блокировать часть потока или пытаться связать консьюмера с неподлинной сессией. Именно поэтому сервер не должен быть единственной стороной, подтверждающей доверенность медиапотока. Проверка должна выполняться на стороне консьюмера независимо от результата обработки данных сервером.

2.4. Временные ограничения и сетевые факторы в медиасистемах

Передача аудио и видеопотоков в реальном времени представляет собой комплексную инженерную задачу, кардинально отличающуюся от традиционной парадигмы файлового обмена данными. Если при передаче статических файлов абсолютным приоритетом является целостность и гарантированная доставка каждого байта информации, что исторически обеспечивается протоколом TCP — Transmission Control Protocol, то для медиаданных критическим фактором становится время. Поточковые системы функционируют в условиях жестких временных ограничений, где задержки, потеря пакетов и вариация задержки оказывают разрушительное воздействие на пользовательский опыт и логику работы декодеров.

Для медиапотока в реальном времени корректность данных не может оцениваться только по факту успешной доставки. Данные могут быть криптографически

корректными, но практически непригодными, если они относятся к устаревшему моменту времени. Поэтому в системе необходимо учитывать не только целостность полезной нагрузки, но и её временную актуальность. Это особенно важно для задач удалённой идентификации, видеонаблюдения, телемедицины и иных сценариев, где получатель должен доверять текущему состоянию источника, а не ранее записанному видеоряду.

2.5. Классификация векторов атак и проблема верификации происхождения данных

Эволюция кибернетических угроз в сфере передачи мультимедийных данных привела к формированию сложных векторов атак, которые выходят далеко за рамки классического перехвата сетевого трафика. Проблема верификации происхождения и аутентичности источника стала одним из главных вызовов десятилетия в связи с экспоненциальным развитием технологий искусственного интеллекта и генеративно-состязательных нейросетей. Разрабатываемая система должна противостоять спектру угроз, которые можно классифицировать по уровню воздействия: сетевой, транспортный и семантический (аппаратный) уровни.

Сетевой уровень включает угрозы перехвата, удаления, задержки, вставки и перестановки сообщений. Транспортный уровень связан с особенностями доставки данных, управлением потерями и сохранением порядка передачи. Семантический или аппаратный уровень отражает более сложную проблему: необходимость доказать, что медиапоток был сформирован доверенным физическим устройством, а не программным эмулятором, виртуальной камерой или генеративной системой.

2.6. Угрозы сетевого уровня и механизмы атак повторного воспроизведения

На сетевом уровне базовыми угрозами являются пассивный перехват трафика, модификация данных в транзите Man-in-the-Middle (MITM, атака посредника), вставка искусственных пакетов, выборочное удаление пакетов и нарушение порядка их следования. В условиях потоковой передачи видео особенно опасной является атака повторного воспроизведения [0]. При реализации данного вектора злоумышленник осуществляет пассивный перехват абсолютно легитимной, криптографически подписанной сессии от аутентичного устройства. Впоследствии этот записанный поток транслируется серверу-маршрутизатору под видом прямой

трансляции реального времени. Поскольку криптографические подписи на каждом кадре остаются математически корректными, базовая проверка электронной цифровой подписи завершается успешно. Подобные атаки успешно применяются для обхода систем контроля доступа и биометрической верификации, когда вместо лица реального человека системе предъявляется его вчерашняя успешная авторизация.

Для противодействия атаке повторного воспроизведения недостаточно проверять только подпись полезной нагрузки. Необходимо связывать каждое сообщение с идентификатором текущей сессии, порядковым номером и временной меткой. В этом случае повторная отправка старого фрагмента может быть обнаружена не по факту нарушения подписи, а по несоответствию контекста: устаревшему времени, уже использованному порядковому номеру или принадлежности к другой сессии.

2.7. Угрозы нарушения порядка, задержки и устаревания данных

Для медиапотока в реальном времени нарушение порядка доставки является самостоятельной угрозой. Даже если каждый отдельный фрагмент полезной нагрузки имеет корректную подпись, изменение последовательности сообщений может привести к некорректному воспроизведению, нарушению логики декодирования или искажению воспринимаемого события. Поэтому в протоколе должна проверяться не только подлинность каждого блока данных, но и монотонность порядковых номеров.

Задержка данных также может использоваться как способ атаки. Нарушитель может не изменять полезную нагрузку и не пытаться подделать подпись, а лишь удерживать корректные сообщения и отправлять их позже. С криптографической точки зрения такие данные могут оставаться валидными, однако с точки зрения медиасистемы они уже не отражают актуальное состояние источника. Для обнаружения такой ситуации в проверяемые служебные поля включается временная метка.

Устаревание данных особенно критично в системах, где видеопоток используется как доказательство текущего присутствия или текущего состояния объекта. Если старый фрагмент потока принимается как новый, нарушается основная цель аутентификации медиаданных. Поэтому проверка актуальности должна рассмат-

риваться как обязательное свойство протокола наряду с проверкой целостности и происхождения.

Отдельную опасность представляет выборочное удаление данных. При такой атаке нарушитель не подделывает сообщения, а исключает часть потока, оставляя только удобные для себя фрагменты. Полностью предотвратить потерю данных на сетевом уровне невозможно, однако анализ порядковых номеров позволяет обнаруживать пропуски и отличать нормальную деградацию сети от систематического вмешательства в поток.

2.8. Дифференциация атак представления и инъекционных атак

Наиболее деструктивным и технологически совершенным классом угроз на современном этапе являются инъекционные атаки на уровне источника генерации контента [0]. Следует строго разграничивать атаки представления и инъекционные атаки. При атаках представления злоумышленник пытается обмануть оптический сенсор камеры, например, показывая камере распечатанную фотографию, силиконовую маску или воспроизводя видео с планшета. Системы защиты от таких атак полагаются на алгоритмы проверки жизнедеятельности, которые анализируют блики экранов, микромимику, текстуру кожи или искажения муара.

Инъекционная атака, напротив, полностью исключает физический оптический сенсор из цепочки захвата данных. Злоумышленник использует специализированное программное обеспечение для создания виртуальных камер, эмуляторы операционных сред, атаки перехвата функций API или аппаратные мосты-эмуляторы для внедрения синтетического цифрового потока непосредственно в ядро операционной системы или драйвер [0]. В результате программный клиент или система получает на вход видеоряд без оптических артефактов, бликов или дефектов освещения, что существенно снижает эффективность классических эвристических алгоритмов анализа изображений.

2.9. Использование генеративного искусственного интеллекта и технологий глубокой подделки в инъекционных векторах

Вектор атаки многократно усиливается доступностью генеративного искусственного интеллекта. Синтетическая генерация контента реального времени, включая глубокие подделки, подмену лиц, перенос мимики и синтез голоса, достиг-

ла качества, неотличимого от реальности [0]. Отчеты показывают, что подавляющее большинство систем верификации, не опирающихся на аппаратные корни доверия, могут быть успешно скомпрометированы качественными инъекциями дипфейков.

Таблица 2.1

Классификация и методы противодействия угрозам мультимедийных данных

Класс атаки	Суть воздействия	Метод реализации	Фокус защиты
Атака представления	Обман объектива камеры	Распечатки, маски, экраны устройств перед камерой	Проверка жизнедеятельности, трёхмерный анализ
Инъекционная атака	Внедрение видео в обход физической камеры	Эмуляторы, виртуальные камеры, перехват программных интерфейсов, глубокая подделка	Аппаратная аттестация, криптографическая подпись на устройстве
Атака повтора	Повторная отправка легитимного старого потока	Перехват трафика с последующей ретрансляцией	Скользящее окно идентификаторов, одноразовые сессионные значения
Сетевая вставка / Модификация	Изменение битов в кадре при передаче	Подмена адресов, перехват маршрутов	Хеширование кадров, защита транспортного уровня
Нарушение порядка и задержка	Передача корректных данных в неправильной последовательности или с опозданием	Удержание сообщений, перестановка, выборочное удаление	Порядковые номера, временные метки, контроль актуальности

2.10. Последствия компрометации сервера-маршрутизатора

Компрометация сервера-маршрутизатора является одним из ключевых сценариев, определяющих необходимость сквозной проверки на стороне коньюмера. Если сервер получает полный доступ к маршрутизации медиапотока, он может попытаться заменить активный поток ранее записанными данными, задержать часть сообщений, нарушить их порядок или передать коньюмеру поток от другого источника. При отсутствии независимой проверки на стороне получателя такая атака могла бы остаться незамеченной.

В рассматриваемой архитектуре сервер выполняет вспомогательную функцию передачи и рассылки медиаданных. Он может проверять сертификат камеры

и поддерживать состояние активной сессии, однако не должен становиться единственной доверенной точкой. Даже если сервер сообщает консюмеру, что поток относится к доверенной камере, консюмер обязан самостоятельно проверить сертификат, результат рукопожатия и подписи полезной нагрузки.

Особенно опасной является ситуация, при которой сервер не изменяет содержимое полезной нагрузки, а манипулирует только порядком и временем доставки. В этом случае цифровые подписи отдельных сообщений могут оставаться корректными, но поток в целом перестаёт отражать реальное состояние источника. Поэтому защита должна включать не только проверку подписи, но и контроль идентификатора сессии, порядкового номера и временной метки.

Такой подход позволяет ограничить последствия компрометации сервера. Скомпрометированный сервер может ухудшить доступность системы, задержать передачу или попытаться нарушить порядок сообщений, однако он не может создать новую корректно подписанную полезную нагрузку от имени доверенной камеры и не может незаметно подменить временный ключ текущей сессии без нарушения проверки рукопожатия.

2.11. Переход к парадигме аппаратного подтверждения происхождения данных

Противодействие синтетической генерации искусственного интеллекта и инъекциям требует переноса парадигмы безопасности с анализа содержания на подтверждение происхождения. Система должна иметь математические доказательства того, что каждый конкретный видеокادر был сформирован на основе фотонов, попавших на матрицу авторизованного устройства захвата, и был немедленно подписан ключом, извлечь который из криптопроцессора невозможно [0].

В рамках такой парадигмы анализ пикселей, поиск признаков монтажа или эвристическая проверка жизнедеятельности не рассматриваются как достаточные механизмы доверия. Эти методы могут использоваться как дополнительные, но они не дают строгого криптографического доказательства происхождения данных. Основным доказательством становится связка между доверенным аппаратным устройством, сертификатом камеры, результатом рукопожатия и подписью полезной нагрузки текущей сессии.

2.12. Выводы по результатам анализа угроз

Проведённый анализ показывает, что защита медиапотока в реальном времени должна учитывать не только классическую модификацию данных при передаче, но и более сложные угрозы: повторное воспроизведение старого потока, задержку корректных сообщений, нарушение порядка данных, компрометацию промежуточного сервера и внедрение синтетического видеоряда в обход физической камеры.

Из рассмотренных угроз следуют основные требования к разрабатываемому протоколу. Во-первых, каждый блок полезной нагрузки должен быть связан с идентификатором сессии, чтобы исключить перенос данных между разными медиасеансами. Во-вторых, порядковый номер должен использоваться для обнаружения повторов, пропусков и нарушения порядка. В-третьих, временная метка должна применяться для проверки актуальности данных. В-четвёртых, подпись полезной нагрузки должна охватывать не только медиаданные, но и служебные поля, определяющие контекст проверки.

Также необходима отдельная проверка происхождения временного ключа, которым подписывается полезная нагрузка текущей сессии. Для этого рукопожатие должно связывать временный открытый ключ с долговременной аппаратной идентичностью камеры, подтверждаемой сертификатом и подписью ключом, размещённым в доверенном платформенном модуле. Такая схема позволяет перенести доверие от анализа содержимого изображения к проверяемому криптографическому происхождению медиапотока.

Следовательно, дальнейшая разработка протокола должна опираться на сочетание аппаратного корня доверия, сессионного рукопожатия, подписи ограниченной по размеру полезной нагрузки и независимой проверки на стороне консюмера. Это позволяет сохранить модель сквозного доверия даже при наличии недоверенной сети и потенциально скомпрометированного сервера-маршрутизатора.

ГЛАВА 3. АНАЛИЗ СУЩЕСТВУЮЩИХ ПОДХОДОВ И ОБОСНОВАНИЕ ВЫБРАННОЙ АРХИТЕКТУРЫ

Перед разработкой собственного протокола необходимо рассмотреть существующие подходы к подтверждению происхождения медиаданных и определить,

какие задачи они решают, а какие ограничения сохраняют. Для данной работы принципиальное значение имеют не только криптографическая защищённость данных, но и возможность проверки происхождения медиапотока в режиме реального времени. Поэтому анализ выполняется с учётом трёх критериев: наличия криптографической привязки данных к источнику, пригодности подхода для потоковой передачи с малой задержкой и возможности выполнять проверку на стороне конечного получателя без полного доверия к серверу.

3.1. Стандартизированные подходы к обеспечению подлинности медиаданных

Стандартизированные подходы направлены на формирование проверяемой цепочки происхождения цифрового контента. Как правило, они связывают медиаданные с набором метаданных, сертификатами и криптографическими подписями. Такой подход хорошо подходит для подтверждения истории создания и изменения файлов, однако его применимость к непрерывному видеопотоку ограничена требованиями к задержке, размеру служебных данных и необходимости частой проверки небольших фрагментов медиапотока.

3.1.1. Индустриальные инициативы по обеспечению подлинности медиаданных

Стремительный рост объёмов манипулированного медиа вызвал активную реакцию индустрии, выразившуюся в формировании ряда стандартов и консорциумов, направленных на внедрение инструментов доказательства подлинности. Наиболее масштабным проектом в этой области выступает стандарт C2PA (Coalition for Content Provenance and Authenticity, коалиция происхождения и подлинности контента), поддерживаемый такими технологическими компаниями, как Adobe, Intel и Microsoft [0].

3.1.2. Архитектура и ограничения стандарта C2PA

Архитектура C2PA опирается на криптографическое связывание контента с манифестами, содержащими утверждения об авторе, устройстве захвата, координатах и истории редактирования. Эти метаданные сериализуются в формате JUMBF (JPEG Universal Metadata Box Format, универсальный блочный формат метаданных

JPEG) и защищаются цепочками сертификатов формата X.509, что позволяет проводить валидацию в автономном режиме без необходимости обращения к централизованному серверу. В теории C2PA предоставляет развитый фундамент доверия. Однако анализ стандарта выявляет ограничения, затрудняющие его применение для потоковых видеоданных в реальном времени с жёсткими временными ограничениями.

3.1.3. Проблема задержек и сегментации в C2PA

Во-первых, архитектура C2PA исторически спроектирована под атомарные статические файловые форматы: JPEG, PNG, MP4 и PDF. Спецификация для потоковых данных в рамках C2PA всё ещё опирается на концепцию сегментации, характерную для протоколов адаптивной потоковой передачи по HTTP (HyperText Transfer Protocol, протокол передачи гипертекста), таких как ISO BMFF (ISO Base Media File Format, базовый формат медиаконтейнера) или CMAF (Common Media Application Format, общий формат медиа-приложений). В рамках этих протоколов медиапоток нарезается на сегменты длительностью от 2 до 10 секунд, и блок криптографического манифеста генерируется и хешируется для всего сегмента целиком. Такой подход вносит задержку, соизмеримую с длиной сегмента. Для сценариев с миллисекундными требованиями к задержке, таких как телемедицина и двусторонняя видеосвязь, ожидание накопления полного сегмента перед проверкой подписи является неудачным решением с точки зрения работы в реальном времени.

Во-вторых, встраивание манифестов в метаданные файлов делает цепочку происхождения хрупкой. На практике при передаче контента через транзитные системы, транскодеры, социальные сети или мессенджеры служебные блоки JUMBF могут удаляться с целью оптимизации размера файла. Это разрушает криптографическое доказательство происхождения, несмотря на то, что сами пиксели кадра могли остаться неизменными. Сама спецификация C2PA указывает, что создание манифеста является вычислительно дорогой операцией и поэтому рекомендует выполнять её как можно реже, только при острой необходимости [0].

Для разрабатываемого протокола это означает, что файловая модель подтверждения происхождения не может быть перенесена на потоковую передачу без существенной переработки. Видеопоток в реальном времени требует проверки небольших единиц данных сразу после получения, а не после накопления крупного сегмента. Кроме того, подтверждение происхождения должно сохраняться даже

при маршрутизации через промежуточный сервер, который не должен становиться единственной доверенной стороной.

3.2. Транспортный уровень защиты медиапотока

Транспортные протоколы решают другую группу задач. Они обеспечивают доставку медиаданных, управление потерями, снижение задержки и защиту канала связи. Однако защита транспортного соединения не тождественна подтверждению происхождения каждого фрагмента медиапотока. Даже если канал зашифрован, конечный получатель должен иметь возможность самостоятельно убедиться, что полезная нагрузка была сформирована доверенным источником и не была подменена на прикладном уровне.

3.2.1. Транспортные протоколы: SRT, WebRTC и QUIC

Альтернативой на транспортном уровне выступают специализированные протоколы доставки, такие как SRT (Secure Reliable Transport, защищённый надёжный транспорт) и WebRTC (Web Real-Time Communication, веб-связь реального времени). Протокол SRT был разработан для надёжной передачи видео по сетям с потерями, с фиксированной задержкой и шифрованием полезной нагрузки. Существуют черновики стандартов по запуску SRT поверх датаграмм QUIC (Quick UDP Internet Connections, быстрые соединения через UDP), что позволяет объединить управление дедлайнами потери пакетов SRT и безопасность TLS 1.3 (Transport Layer Security, защита транспортного уровня) от QUIC [0]. Однако спецификация SRT ограничена защитой канала связи. Как отмечается в анализе протоколов, SRT не предоставляет внутренних механизмов верификации криптографической целостности самих пакетов на уровне приложения и не отличает легитимный пакет от внедрённого, если злоумышленник скомпрометировал узел отправки. WebRTC, обеспечивающий минимальную задержку связи, также сфокусирован на шифровании и допускает передачу служебных данных, однако не задаёт стандартизованного механизма аппаратно подтверждаемого происхождения медиаконтента с проверкой подписи на уровне отдельных фрагментов в архитектуре публикатор-подписчик.

Традиционно транспортным базисом для мультимедиа выступал стек RTP (Real-time Transport Protocol, протокол передачи данных в реальном времени) или UDP (User Datagram Protocol, протокол пользовательских датаграмм), однако в

современных высоконагруженных системах произошёл сдвиг в сторону протокола QUIC. В отличие от TCP (Transmission Control Protocol, протокол управления передачей), QUIC устраняет проблему блокировки начала очереди. При использовании TCP потеря одного сегмента приводит к остановке доставки всех последующих данных до успешной ретрансмиссии. В протоколе QUIC данные мультиплексируются в независимые логические потоки. Потеря пакета в одном потоке не влияет на обработку данных в параллельных потоках, что важно для передачи медиаданных в реальном времени [0].

Таблица 3.1

Сравнительная характеристика транспортных протоколов

Характеристика протокола	TCP	UDP/RTP	QUIC
Доставка	Абсолютно надёжная	Ненадёжная	Частично надёжная, настраиваемая
Блокировка очереди	Присутствует	Отсутствует	Отсутствует за счёт изоляции потоков
Отношение к устаревшим данным	Ретранслирует до успешной доставки	Допускает потерю	Позволяет сбрасывать поток по дедлайну
Криптография транспорта	Внешняя, через TLS	Внешняя, через SRTP или DTLS	Встроенная, через TLS 1.3

Интеграция криптографического протокола поверх такой транспортной модели требует обеспечения атомарности проверок: криптографическая метка должна генерироваться не для потока в целом, а для отдельных ограниченных по размеру блоков полезной нагрузки. Это необходимо для того, чтобы потеря одного фрагмента не разрушала криптографический контекст последующих данных, как это может происходить в схемах, завязанных на непрерывное состояние всего потока.

Следовательно, транспортные протоколы целесообразно рассматривать не как замену прикладной криптографической проверке, а как основу для доставки данных. Они могут обеспечить низкую задержку, мультиплексирование, управление потерями и защищённый канал передачи, но не формируют самостоятельного доказательства того, что конкретный фрагмент медиапотока был создан доверенной

камерой. В рассматриваемой задаче транспортный уровень должен быть дополнен прикладным механизмом подписи и проверки полезной нагрузки.

3.3. Криптографические примитивы

Для достижения заявленных целей и обеспечения устойчивости медиапотока к описанным векторам атак архитектура должна опираться на надёжные криптографические примитивы. Интеграция таких примитивов в высоконагруженный конвейер реального времени требует анализа их вычислительной сложности, размеров генерируемых артефактов и уязвимостей. Протокол опирается на комбинирование криптографических хеш-функций и алгоритмов электронной цифровой подписи.

3.3.1. Хеш-функции как механизм обеспечения целостности

Криптографическая хеш-функция выступает одним из механизмов защиты целостности. Она отображает массив данных в дайджест фиксированной длины. Лавинный эффект хеш-функций гарантирует, что изменение хотя бы одного бита во входных данных приведёт к формированию нового дайджеста, что будет обнаружено при верификации. Выбор между алгоритмами семейства SHA-2 (Secure Hash Algorithm, безопасный алгоритм хеширования), в частности SHA-256 и SHA-512, важен для производительности систем, обрабатывающих большие объёмы медиаданных.

3.3.2. Сравнительный анализ SHA-256 и SHA-512

Хотя интуитивно может показаться, что алгоритм с меньшим размером выхода, то есть SHA-256, должен работать быстрее, архитектурные особенности современных 64-битных центральных процессоров диктуют иное. Алгоритм SHA-256 оперирует 32-битными машинными словами и выполняет 64 раунда функции компрессии над блоками данных размером 512 бит. Алгоритм SHA-512, напротив, был спроектирован для 64-битных регистров: он выполняет 80 раундов компрессии, но обрабатывает входные данные блоками по 1024 бита. Как показывают бенчмарки на современных x86-64 и ARM64 системах, пропускная способность SHA-512 достигает 315–332 MiB/s, в то время как SHA-256 выдаёт около 200–206 MiB/s. Таким образом, на 64-битных серверах и аппаратных плат-

формах SHA-512 работает на 50–66% быстрее, что делает его кандидатом для хеширования в реальном времени [0].

3.3.3. Необходимость асимметричной криптографии

Тем не менее хеш не защищает от направленной подделки: злоумышленник может подменить полезную нагрузку и пересчитать хеш. Для решения задачи аутентификации источника требуется применение асимметричной криптографии. В индустрии доминируют три базовых стандарта: RSA (Rivest–Shamir–Adleman, алгоритм Ривеста, Шамира и Адлемана), ECDSA (Elliptic Curve Digital Signature Algorithm, алгоритм цифровой подписи на эллиптических кривых) и EdDSA (Edwards-curve Digital Signature Algorithm, алгоритм цифровой подписи на кривых Эдвардса), в частности Ed25519 [0].

3.3.4. Алгоритм RSA и его ограничения

Алгоритм RSA исторически широко распространён. Его математическая модель основана на сложности факторизации больших полупростых чисел. В рамках разрабатываемого протокола RSA выбран не для подписи каждого фрагмента медиапотока, а для подтверждения долговременной идентичности камеры на этапе рукопожатия. Такой выбор обусловлен высокой совместимостью RSA с сертификатами X.509, инфраструктурой Root CA (Root Certificate Authority, корневой центр сертификации), а также с доверенным платформенным модулем, внутри которого может храниться долговременный закрытый ключ устройства.

Сильной стороной RSA является высокая скорость проверки подписи. При использовании малой публичной экспоненты верификация RSA-2048 занимает около ~ 0.03 миллисекунды, что делает алгоритм удобным для проверки контекста рукопожатия на стороне сервера и консюмера. В проекте RSA-2048 используется для подписи контекста рукопожатия по схеме RSA-PSS + SHA-256, а заводской корневой центр сертификации использует RSA-4096 для выпуска сертификатов камер.

При этом RSA имеет ограничения при работе с потоковым видео. Генерация подписи на стороне камеры требует больше вычислительных ресурсов и может занимать более 1.2 мс, что делает RSA неудачным выбором для подписи каждого фрагмента медиапотока в режиме реального времени. Кроме того, размер подписи составляет 256 байт для RSA-2048 и 512 байт для RSA-4096, что существенно

увеличило бы служебную нагрузку при передаче каждого блока данных. Поэтому в проекте RSA применяется только для доверенного установления сессии, а для подписи полезной нагрузки текущей сессии используется более компактный и быстрый алгоритм Ed25519.

3.3.5. ECDSA и проблема случайных чисел

Переход к эллиптической криптографии решает проблему размеров. Алгоритм ECDSA на кривых Вейерштрасса генерирует подписи размером всего 64 байта, обеспечивая уровень безопасности, эквивалентный RSA-3072. Вычислительные задержки также сбалансированы: генерация занимает ~ 0.026 мс, верификация ~ 0.057 мс [0]. Однако ECDSA имеет критическую архитектурную уязвимость, связанную со случайными числами, используемыми при каждой операции подписания. Если генератор псевдослучайных чисел аппаратного модуля работает некорректно, предсказуемо или генерирует коллизии, злоумышленник, наблюдая две подписи, может вычислить закрытый ключ устройства, полностью компрометируя корень доверия. Таким образом, несмотря на компактный размер подписи и высокую скорость работы, ECDSA менее предпочтителен для встроенных камер из-за зависимости безопасности от качества случайного числа.

3.3.6. Ed25519 как базовое решение

Решением, положенным в основу проектируемого протокола, является алгоритм Ed25519, то есть схема цифровой подписи Эдвардса, функционирующая на эллиптической кривой Curve25519 [0]. Главное преимущество Ed25519 заключается в том, что это детерминированная подпись. При формировании подписи не используется аппаратный генератор случайных чисел. Одноразовый секрет вычисляется детерминированно как хеш SHA-512 от конкатенации закрытого ключа и самого сообщения. Это математически исключает утечку приватного ключа из-за некачественного генератора случайных чисел. В тестах на архитектурах ARM и x86 алгоритм Ed25519 показывает высокую скорость генерации подписи: наложение подписи занимает около ~ 0.020 миллисекунды, а верификация выполняется за ~ 0.045 мс [0]. Более того, операции на кривой Эдвардса спроектированы так, что они выполняются за константное время, что снижает риск атак по сторонним каналам, в частности тайминг-атак, когда злоумышленник пытается извлечь ключ, измеряя время отклика процессора. Размер подписи и ключа составляет 64 байта и

32 байта соответственно, что соответствует требованиям минимизации служебной нагрузки для транспортного уровня QUIC.

3.4. Аппаратный корень доверия и использование доверенного платформенного модуля

3.4.1. Понятие аппаратного корня доверия

Аппаратный корень доверия представляет собой защищённый аппаратный компонент, на который опирается вся последующая цепочка проверки подлинности устройства. Его основная задача заключается в том, чтобы хранить критически важные криптографические секреты вне обычной программной среды. В отличие от ключей, размещённых в файлах, памяти процесса или конфигурации приложения, ключи внутри аппаратного модуля не должны покидать защищённую область устройства. Это снижает риск их извлечения при компрометации операционной системы, приложения или пользовательского окружения.

3.4.2. Назначение доверенного платформенного модуля

В рамках данной работы в качестве аппаратного корня доверия используется TPM (Trusted Platform Module, доверенный платформенный модуль). Он представляет собой специализированный криптографический модуль, предназначенный для безопасной генерации, хранения и использования ключей. Такой модуль может выполнять криптографические операции внутри себя: например, формировать подпись с использованием закрытого ключа, не раскрывая сам закрытый ключ вызывающей программе. Приложение передаёт в доверенный платформенный модуль данные или их хеш, а модуль возвращает результат операции подписи. Таким образом, программная часть системы получает возможность доказать владение закрытым ключом, но не получает доступ к самому ключу.

3.4.3. Обоснование использования доверенного платформенного модуля для подтверждения происхождения медиапотока

Использование доверенного платформенного модуля особенно важно для задачи подтверждения происхождения медиапотока. Если закрытый ключ камеры хранится как обычный файл, злоумышленник при компрометации устройства

может скопировать его и использовать для создания ложного видеопотока на другом оборудовании. В этом случае криптографическая подпись формально будет корректной, но она уже не будет доказывать связь потока с доверенным аппаратным источником. При использовании доверенного платформенного модуля закрытый ключ создаётся и хранится внутри защищённого аппаратного компонента, поэтому его извлечение существенно затрудняется. Это позволяет связать долговременную идентичность камеры не только с программным клиентом, но и с конкретным аппаратным устройством.

3.4.4. Использование доверенного платформенного модуля на этапе рукопожатия

В проектируемом протоколе доверенный платформенный модуль используется не для подписи каждого фрагмента медиапотока, а для подтверждения доверенной сессии на этапе рукопожатия. Долговременный закрытый ключ камеры RSA-2048 генерируется внутри доверенного платформенного модуля и не покидает устройство. Соответствующий открытый ключ включается в сертификат X.509 камеры, который подписывается заводским корневым центром сертификации RSA. Благодаря этому консюмер может проверить, что полученный открытый ключ действительно принадлежит доверенному устройству, выпущенному производителем.

Во время рукопожатия камера формирует временную пару ключей Ed25519, предназначенную для подписи полезной нагрузки текущей сессии. Затем камера подписывает контекст рукопожатия долговременным закрытым ключом, размещённым в доверенном платформенном модуле. В подписываемый контекст входят идентификатор сессии, одноразовое случайное значение консюмера, временная метка и временный открытый ключ Ed25519. Такая схема связывает временный ключ подписи полезной нагрузки с долговременной аппаратной идентичностью камеры. Консюмер, проверяя сертификат камеры и подпись контекста рукопожатия, получает доказательство того, что временный ключ Ed25519 был выдан именно доверенным устройством, а не подставлен злоумышленником.

3.4.5. Разделение ролей долговременного и временного ключей

Разделение ролей между RSA-ключом в доверенном платформенном модуле и временным ключом Ed25519 обусловлено требованиями режима реального времени. Операции RSA с использованием доверенного платформенного модуля

обладают более высокой стоимостью и не подходят для подписи каждого фрагмента медиапотока. Поэтому RSA-ключ используется только для редкой, но критически важной операции — установления доверенной сессии. После успешного рукопожатия подпись полезной нагрузки выполняется временным ключом Ed25519, созданным в оперативной памяти. Это снижает задержки и накладные расходы, сохраняя при этом связь с аппаратным источником доверия.

3.5. Формализация требований к протоколу

Анализ предметной области и векторов угроз позволяет сформулировать набор требований к разрабатываемому протоколу криптографической аутентификации медиапотока. Эти требования должны обеспечивать защиту от подмены данных в реальном времени при сохранении высокой производительности системы.

3.5.1. Аппаратный корень доверия

Ключевое требование для защиты от инъекционных атак состоит в том, что долговременный закрытый ключ устройства, используемый для подписи контекста рукопожатия, не должен существовать в оперативной памяти компьютера или смартфона в открытом виде. Он должен генерироваться и безопасно храниться в изолированных аппаратных модулях, таких как доверенный платформенный модуль, или в защищённых анклавах процессоров TEE (Trusted Execution Environment, доверенная среда выполнения), например Intel TDX или AMD SNP. Протокол должен поддерживать проверку сертификата камеры и подписи контекста рукопожатия, чтобы консюмер мог криптографически убедиться, что временный ключ текущей сессии связан с доверенным устройством-источником. Для этого используется механизм обмена сертификатами, подписанными доверенными центрами сертификации производителей оборудования [0].

3.5.2. Устойчивость к активным манипуляциям и атакам повтора

Протокол должен обеспечивать обнаружение несанкционированных изменений подписанной полезной нагрузки и связанных с ней служебных полей. К таким полям относятся идентификатор сессии, порядковый номер полезной нагрузки и временная метка. Любое изменение полезной нагрузки, идентификатора сессии,

порядкового номера или временной метки должно приводить к ошибке проверки цифровой подписи.

Для противодействия атакам повторного воспроизведения архитектура должна включать одноразовые случайные значения на этапе рукопожатия и строгую проверку монотонного возрастания порядковых номеров. Это позволяет отклонять повторно переданные, устаревшие или относящиеся к другой сессии фрагменты медиапотока.

При этом криптографический уровень не должен зависеть от внутренней структуры видеокodeка. В текущей реализации внутри полезной нагрузки передаются данные MPEG-TS, содержащие H.264-видео, однако сам протокол рассматривает полезную нагрузку как ограниченный по размеру блок данных и не выполняет разбор структуры codeка.

3.5.3. Минимальные накладные расходы для режима реального времени

Время формирования криптографической подписи полезной нагрузки на стороне устройства-источника не должно превышать 1 миллисекунды, а процедура верификации на стороне консюмера должна выполняться за микросекунды. Суммарная алгоритмическая задержка не должна нарушать плавность воспроизведения потока 60 FPS (Frames Per Second, кадров в секунду), где бюджет времени на кадр составляет 16.6 мс. Размер служебной метки, включающей заголовок и подпись, должен быть минимальным, чтобы не провоцировать излишнюю фрагментацию IP-пакетов (Internet Protocol, межсетевой протокол).

3.5.4. Масштабируемость сервера-маршрутизатора

Хранение публичных ключей и состояний активных сессий должно выполняться в оперативной памяти с использованием конкурентных ассоциативных массивов. Это позволяет исключить задержки, связанные с обращением к реляционным базам данных, дисковым вводом-выводом и блокировками транзакций, обеспечивая доступ к данным с асимптотической сложностью $O(1)$.

3.5.5. Сквозная проверка на стороне консюмера

Архитектура не должна предполагать, что консюмер слепо доверяет серверу. Сервер выступает маршрутизатором, но валидация подлинности должна дублироваться на стороне конечного потребителя видео. Такое требование соот-

ветствует модели нулевого доверия, при которой промежуточные узлы не получают полномочий окончательно подтверждать достоверность медиаданных.

3.6. Итог главы

Таким образом, существующие решения не в полной мере закрывают задачу криптографической валидации источника на уровне отдельного фрагмента медиапотока в режиме реального времени. Файловые подходы, такие как C2PA, ориентированы преимущественно на постфактум-проверку медиаконтента и требуют накопления крупных фрагментов данных, что плохо согласуется с требованиями низкой задержки. Транспортные протоколы, такие как SRT и QUIC, обеспечивают защиту канала передачи, однако не решают задачу аппаратно подтверждаемого происхождения каждого подписанного блока данных и его устойчивости к внедрению на уровне приложения. Это обосновывает необходимость разработки гибридного решения, работающего поверх QUIC и реализующего гранулярную проверку подписанной полезной нагрузки с аппаратной привязкой источника.

Гибридный подход позволяет разделить ответственность между уровнями системы. Транспортный уровень отвечает за доставку медиаданных с учётом задержек и потерь, а прикладной криптографический уровень отвечает за проверку происхождения, целостности, актуальности и принадлежности полезной нагрузки конкретной сессии. Такое разделение соответствует архитектуре разрабатываемого протокола, в котором сервер выполняет маршрутизацию потока, а окончательное решение о доверии к данным принимает консюмер на основании сертификата камеры, результата рукопожатия и подписи полезной нагрузки.

ГЛАВА 4. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ РАЗРАБОТАННОГО РЕШЕНИЯ

4.1. Сценарий работы разработанного приложения

Разработанное приложение реализует прототип протокола непрерывной криптографической аутентификации медиапотока в реальном времени. В рамках демонстрационного сценария система включает три программных компонента: продюсер, сервер-маршрутизатор и консюмер. Продюсер имитирует доверенный

источник медиаданных, формирует кадры и подписывает их временным ключом текущей сессии. Сервер-маршрутизатор принимает данные от продюсера и передаёт их консюмеру, не выполняя окончательную проверку подлинности каждого кадра. Консюмер получает данные, проверяет сертификат камеры, результат рукопожатия и подписи кадров.

Общая схема работы разработанного приложения представлена на рисунке 4.1. На ней отражена последовательность взаимодействия основных компонентов: запуск сервера-маршрутизатора, подключение продюсера, подключение консюмера, выполнение рукопожатия и последующая передача кадров.

Общая схема работы разработанного приложения

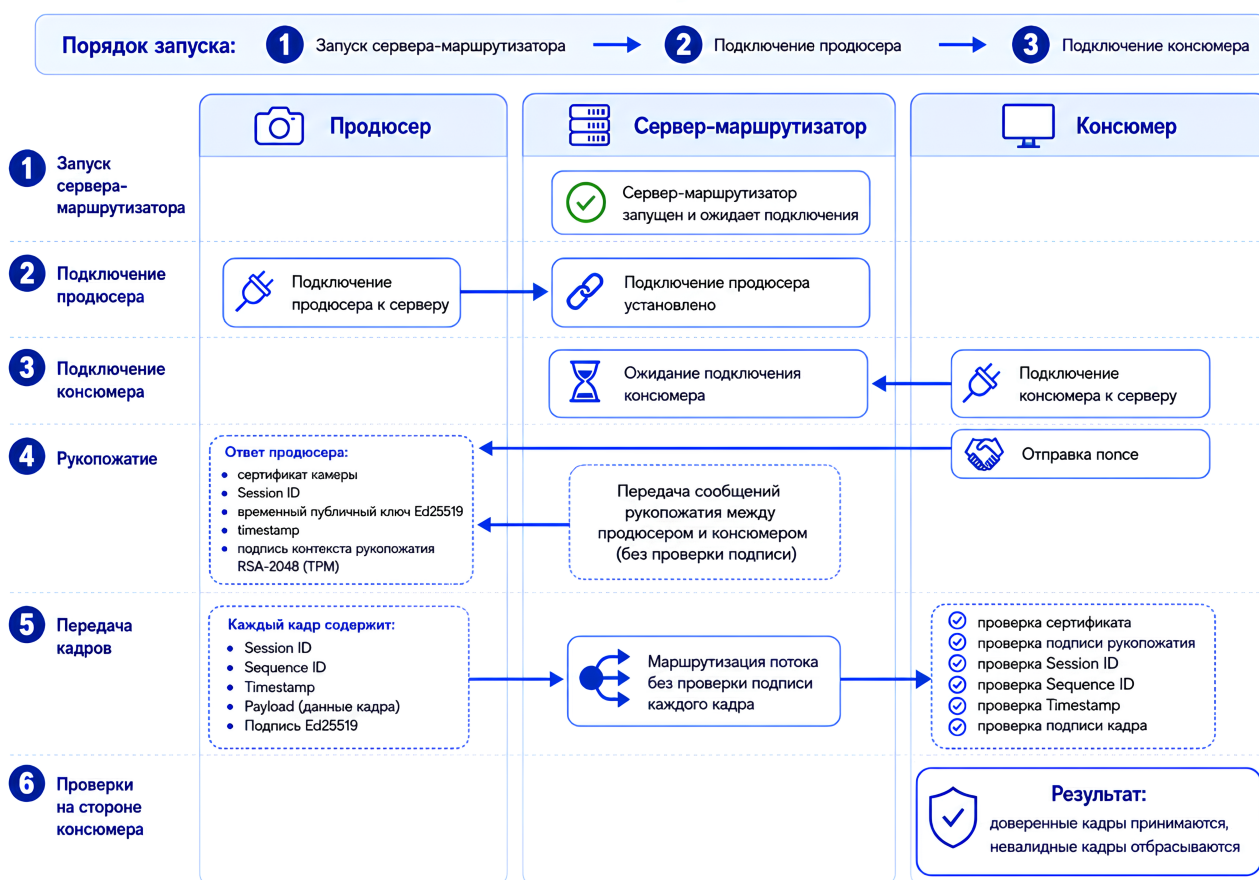


Рис.4.1. Сценарий работы разработанного приложения

Работа приложения начинается с запуска сервера-маршрутизатора. После этого запускается продюсер, который подготавливает криптографический контекст: использует долговременный ключ камеры для подтверждения подписи доверенной сессии и формирует временную пару ключей Ed25519 для подписи кадров. Затем запускается консюмер, инициирующий процедуру рукопожатия. В ходе этой процедуры консюмер передаёт одноразовое случайное значение, а продюсер возвращает

сертификат камеры, идентификатор сессии, временный публичный ключ и подпись контекста рукопожатия.

После успешного рукопожатия начинается передача кадров. Каждый кадр содержит полезную нагрузку, идентификатор сессии, порядковый номер, временную метку и цифровую подпись. Консюмер проверяет, что кадр относится к активной сессии, не является повторным, не является просроченным и имеет корректную подпись. Если хотя бы одно из условий нарушается, кадр отбрасывается и не передаётся дальше в обработку.

Таким образом, демонстрационный сценарий показывает основную идею разработанного протокола: сервер участвует в маршрутизации медиапотока, но окончательное решение о доверии к данным принимает консюмер. Это соответствует модели сквозной проверки, при которой промежуточный узел не является единственной доверенной стороной.

4.2. Демонстрация установления доверенной сессии

Установление доверенной сессии выполняется на этапе рукопожатия между консюмером и продюсером через сервер-маршрутизатор. Цель данного этапа заключается в том, чтобы связать временный ключ подписи кадров с долговременной аппаратной идентичностью камеры. Для защиты от повторного воспроизведения ранее записанных сообщений консюмер формирует одноразовое случайное значение, которое включается в подписываемый контекст рукопожатия.

На рисунке 4.2 показан вывод консюмера при создании запроса рукопожатия и последующей проверке ответа. В журнале отображается сформированное значение `consumer_nonce`, после чего фиксируется получение проверенного ответа: идентификатор сессии, субъект сертификата камеры и сообщение об успешном завершении рукопожатия. Это подтверждает, что консюмер принял параметры доверенной сессии только после выполнения необходимых проверок.

```

PS D:\GolangProject\Диплом 2026\crypto-stream-auth> go run ./cmd/consumer/main.go
2026/05/23 14:16:00 handshake request created:
  protocol_version=1
  consumer_nonce=82acfdb64dec5f3abbbc37144148355
2026/05/23 14:16:00 handshake response received:
  session_id=66191fe10b20cc37deaa43fdf63b9922
  certificate_sha256=de6acca1cc94a0a5fb691579d1ceb7cbd61b67c1b4c36edf324d7a7036e62ede
  ephemeral_public_key=f11b8e5f265f55b04bd090d1a5eb150eed90a67c14232a8a9d560f3673fc50f1
  timestamp=1779534960
  signature_bytes=256
2026/05/23 14:16:00 handshake response verified:
  session_id=66191fe10b20cc37deaa43fdf63b9922
  camera_subject="cam-001"
2026/05/23 14:16:00 handshake ok:

```

Рис.4.2. Создание запроса и проверка ответа рукопожатия на стороне консюмера

На рисунке 4.3 представлен вывод продюсера. Продюсер принимает запрос рукопожатия от консюмера, получает версию протокола и одноразовое случайное значение, после чего формирует ответ. В ответ включаются идентификатор сессии, хеш сертификата камеры, временный публичный ключ Ed25519, временная метка и подпись контекста рукопожатия. Размер подписи составляет 256 байт, что соответствует использованию RSA-2048 для подтверждения доверенной идентичности камеры.

```

PS D:\GolangProject\Диплом 2026\crypto-stream-auth> go run ./cmd/producer/main.go
2026/05/23 14:15:51 producer listening on 127.0.0.1:4242
2026/05/23 14:16:00 handshake request received:
  protocol_version=1
  consumer_nonce=82acfdb64dec5f3abbbc37144148355
2026/05/23 14:16:00 handshake response built:
  session_id=66191fe10b20cc37deaa43fdf63b9922
  certificate_subject="cam-001"
  certificate_sha256=de6acca1cc94a0a5fb691579d1ceb7cbd61b67c1b4c36edf324d7a7036e62ede
  ephemeral_public_key=f11b8e5f265f55b04bd090d1a5eb150eed90a67c14232a8a9d560f3673fc50f1
  timestamp=1779534960
  signature_bytes=256
2026/05/23 14:16:00 handshake ok:

```

Рис.4.3. Формирование ответа рукопожатия на стороне продюсера

На рисунке 4.4 показан вывод сервера-маршрутизатора. Сервер принимает поток от продюсера, получает запрос рукопожатия от консюмера, передаёт его продюсеру и затем маршрутизирует ответ обратно консюмеру. Дополнительно сервер проверяет сертификат камеры по доверенному корневому сертификату и фиксирует успешную маршрутизацию рукопожатия. При этом сервер не становится единственной доверенной стороной, поскольку окончательная проверка параметров доверенной сессии выполняется на стороне консюмера.

```

PS D:\GolangProject\Диплом 2026\crypto-stream-auth> go run ./cmd/server/main.go
2026/05/23 14:15:47 server listening on 127.0.0.1:4343
2026/05/23 14:15:47 producer upstream: 127.0.0.1:4242
2026/05/23 14:16:00 handshake request from consumer:
  protocol_version=1
  consumer_nonce=82acfdb64dec5f3abbbc37144148355
2026/05/23 14:16:00 handshake response from producer:
  session_id=66191fe10b20cc37deaa43fdf63b9922
  certificate_sha256=de6accalcc94a0a5fb691579d1ceb7cbd61b67c1b4c36edf324d7a7036e62ede
  ephemeral_public_key=f11b8e5f265f55b04bd090d1a5eb150eed90a67c14232a8a9d560f3673fc50f1
  timestamp=1779534960
  signature_bytes=256
2026/05/23 14:16:00 camera certificate verified by server:
  certificate_sha256=de6accalcc94a0a5fb691579d1ceb7cbd61b67c1b4c36edf324d7a7036e62ede
2026/05/23 14:16:00 handshake routed

```

Рис.4.4. Маршрутизация рукопожатия через сервер

Таким образом, представленные результаты демонстрируют полный цикл установления доверенной сессии. Консюмер создаёт запрос с одноразовым случайным значением, продюсер формирует подписанный ответ с параметрами сессии, а сервер выполняет маршрутизацию сообщений между участниками. Успешное завершение рукопожатия означает, что временный публичный ключ Ed25519 связан с сертификатом камеры и долговременным ключом RSA-2048. После этого дальнейшая проверка кадров может выполняться с использованием временного ключа текущей сессии, а долговременный ключ применяется только для установления доверия на начальном этапе.

4.3. Демонстрация передачи и проверки кадров

После установления доверенной сессии продюсер начинает формировать и передавать кадры медиапотока. В демонстрационном сценарии кадр рассматривается как ограниченная по размеру полезная нагрузка, не требующая обязательного разбора внутренней структуры видеокодека. Такой подход позволяет использовать протокол независимо от конкретного формата кодирования видео.

На рисунке 4.5 представлен вывод продюсера при передаче медиапотока. В журнале отображается статистика формирования кадров: количество переданных кадров за интервал, объём переданных данных, частота кадров и средний размер полезной нагрузки. Значения fps находятся около 30 кадров в секунду, что подтверждает работу приложения в режиме потоковой передачи, близком к реальному времени.

```

2026/05/23 14:16:01 producer stats: frames=1 bytes=27717 fps=1.0 avg_payload=27717
2026/05/23 14:16:02 producer stats: frames=32 bytes=78690 fps=31.3 avg_payload=2459
2026/05/23 14:16:03 producer stats: frames=31 bytes=73348 fps=30.3 avg_payload=2366
2026/05/23 14:16:04 producer stats: frames=31 bytes=67803 fps=30.2 avg_payload=2187
2026/05/23 14:16:05 producer stats: frames=30 bytes=64239 fps=29.8 avg_payload=2141
2026/05/23 14:16:06 producer stats: frames=31 bytes=71595 fps=30.3 avg_payload=2309
2026/05/23 14:16:07 producer stats: frames=30 bytes=69318 fps=29.8 avg_payload=2310
2026/05/23 14:16:08 producer stats: frames=31 bytes=74083 fps=30.3 avg_payload=2389
2026/05/23 14:16:09 producer stats: frames=31 bytes=65870 fps=30.3 avg_payload=2124
2026/05/23 14:16:10 producer stats: frames=30 bytes=60804 fps=29.8 avg_payload=2026
2026/05/23 14:16:11 producer stats: frames=31 bytes=65717 fps=30.3 avg_payload=2119
2026/05/23 14:16:12 producer stats: frames=30 bytes=71240 fps=29.8 avg_payload=2374
2026/05/23 14:16:13 producer stats: frames=31 bytes=69576 fps=30.3 avg_payload=2244
2026/05/23 14:16:14 producer stats: frames=31 bytes=66045 fps=30.3 avg_payload=2130
2026/05/23 14:16:15 producer stats: frames=30 bytes=70333 fps=29.7 avg_payload=2344
2026/05/23 14:16:16 producer stats: frames=31 bytes=67099 fps=30.3 avg_payload=2164
2026/05/23 14:16:17 producer stats: frames=30 bytes=77999 fps=29.8 avg_payload=2599
2026/05/23 14:16:18 producer stats: frames=31 bytes=104390 fps=30.3 avg_payload=3367
2026/05/23 14:16:19 producer stats: frames=31 bytes=105056 fps=30.3 avg_payload=3388
2026/05/23 14:16:20 producer stats: frames=30 bytes=101312 fps=29.8 avg_payload=3377
2026/05/23 14:16:21 producer stats: frames=31 bytes=114932 fps=30.3 avg_payload=3707
2026/05/23 14:16:22 producer stats: frames=30 bytes=80503 fps=29.7 avg_payload=2683
2026/05/23 14:16:23 producer stats: frames=31 bytes=83191 fps=30.3 avg_payload=2683
2026/05/23 14:16:24 producer stats: frames=31 bytes=77844 fps=30.3 avg_payload=2511
2026/05/23 14:16:25 producer stats: frames=30 bytes=79542 fps=29.8 avg_payload=2651
2026/05/23 14:16:26 producer stats: frames=31 bytes=84702 fps=30.3 avg_payload=2732
2026/05/23 14:16:27 producer stats: frames=29 bytes=71943 fps=28.8 avg_payload=2480
2026/05/23 14:16:28 producer stats: frames=31 bytes=73284 fps=30.3 avg_payload=2364

```

Рис.4.5. Передача кадров медиапотока на стороне продюсера

Перед отправкой кадра продюсер формирует сообщение для подписи. В него входят идентификатор сессии, порядковый номер кадра, временная метка и полезная нагрузка. Затем сообщение подписывается временным закрытым ключом Ed25519, созданным для текущей сессии. Полученная подпись добавляется к кадру, после чего кадр передаётся через сервер-маршрутизатор консюмеру.

На рисунке 4.6 показан вывод сервера-маршрутизатора при пересылке кадров. Сервер фиксирует количество маршрутизированных кадров, объём переданных байт, частоту кадров и средний размер полезной нагрузки. Эти данные демонстрируют, что сервер выполняет роль промежуточного узла передачи и не осуществляет криптографическую проверку подписи каждого кадра.


```

2026/05/23 14:16:02 server relay stats: frames=33 bytes=106407 fps=32.2 avg_payload=3224
2026/05/23 14:16:03 server relay stats: frames=31 bytes=73348 fps=30.3 avg_payload=2366
2026/05/23 14:16:04 server relay stats: frames=31 bytes=67803 fps=30.2 avg_payload=2187
2026/05/23 14:16:05 server relay stats: frames=30 bytes=64239 fps=29.8 avg_payload=2141
2026/05/23 14:16:06 server relay stats: frames=31 bytes=71595 fps=30.3 avg_payload=2309
2026/05/23 14:16:07 server relay stats: frames=30 bytes=69318 fps=29.8 avg_payload=2310
2026/05/23 14:16:08 server relay stats: frames=31 bytes=74083 fps=30.3 avg_payload=2389
2026/05/23 14:16:09 server relay stats: frames=31 bytes=65870 fps=30.3 avg_payload=2124
2026/05/23 14:16:10 server relay stats: frames=30 bytes=60804 fps=29.8 avg_payload=2026
2026/05/23 14:16:11 server relay stats: frames=31 bytes=65717 fps=30.3 avg_payload=2119
2026/05/23 14:16:12 server relay stats: frames=30 bytes=71240 fps=29.8 avg_payload=2374
2026/05/23 14:16:13 server relay stats: frames=31 bytes=69576 fps=30.3 avg_payload=2244
2026/05/23 14:16:14 server relay stats: frames=31 bytes=66045 fps=30.3 avg_payload=2130
2026/05/23 14:16:15 server relay stats: frames=30 bytes=70333 fps=29.7 avg_payload=2344
2026/05/23 14:16:16 server relay stats: frames=31 bytes=67099 fps=30.3 avg_payload=2164
2026/05/23 14:16:17 server relay stats: frames=30 bytes=77999 fps=29.8 avg_payload=2599
2026/05/23 14:16:18 server relay stats: frames=31 bytes=104390 fps=30.3 avg_payload=3367
2026/05/23 14:16:19 server relay stats: frames=31 bytes=105056 fps=30.3 avg_payload=3388
2026/05/23 14:16:20 server relay stats: frames=30 bytes=101312 fps=29.8 avg_payload=3377
2026/05/23 14:16:21 server relay stats: frames=31 bytes=114932 fps=30.3 avg_payload=3707
2026/05/23 14:16:22 server relay stats: frames=30 bytes=80503 fps=29.8 avg_payload=2683

```

Рис.4.6. Маршрутизация кадров медиапотока через сервер

Сервер-маршрутизатор в данном процессе выполняет приём и пересылку данных между участниками. Такой подход снижает вычислительную нагрузку на сервер и позволяет сохранить его роль как быстрого транспортного узла. При этом сервер не получает возможности незаметно изменить содержимое кадра, поскольку окончательная проверка выполняется на стороне консюмера.

На рисунке 4.7 представлен вывод консюмера при получении и проверке кадров. В журнале отображаются принятые кадры, отброшенные кадры, количество кадров с некорректной подписью, объём полученных данных, частота кадров и средний размер полезной нагрузки. В приведённом примере значения `dropped=0` и `bad_signatures=0` показывают, что в ходе демонстрации кадры успешно проходили проверку и не отбрасывались как недоверенные.

```

2026/05/23 14:16:00 playing live video stream
2026/05/23 14:16:02 consumer stats: accepted=33 dropped=0 bad_signatures=0 bytes=106407 fps=32.3 avg_payload=3224
2026/05/23 14:16:03 consumer stats: accepted=31 dropped=0 bad_signatures=0 bytes=73348 fps=30.3 avg_payload=2366
2026/05/23 14:16:04 consumer stats: accepted=31 dropped=0 bad_signatures=0 bytes=67803 fps=30.3 avg_payload=2187
2026/05/23 14:16:05 consumer stats: accepted=30 dropped=0 bad_signatures=0 bytes=64239 fps=29.8 avg_payload=2141
2026/05/23 14:16:06 consumer stats: accepted=31 dropped=0 bad_signatures=0 bytes=71595 fps=30.3 avg_payload=2309
2026/05/23 14:16:07 consumer stats: accepted=30 dropped=0 bad_signatures=0 bytes=69318 fps=29.8 avg_payload=2310
2026/05/23 14:16:08 consumer stats: accepted=31 dropped=0 bad_signatures=0 bytes=74083 fps=30.3 avg_payload=2389
2026/05/23 14:16:09 consumer stats: accepted=31 dropped=0 bad_signatures=0 bytes=65870 fps=30.3 avg_payload=2124
2026/05/23 14:16:10 consumer stats: accepted=30 dropped=0 bad_signatures=0 bytes=60804 fps=29.7 avg_payload=2026
2026/05/23 14:16:11 consumer stats: accepted=31 dropped=0 bad_signatures=0 bytes=65717 fps=30.3 avg_payload=2119
2026/05/23 14:16:12 consumer stats: accepted=30 dropped=0 bad_signatures=0 bytes=71240 fps=29.8 avg_payload=2374
2026/05/23 14:16:13 consumer stats: accepted=31 dropped=0 bad_signatures=0 bytes=69576 fps=30.3 avg_payload=2244
2026/05/23 14:16:14 consumer stats: accepted=31 dropped=0 bad_signatures=0 bytes=66045 fps=30.3 avg_payload=2130
2026/05/23 14:16:15 consumer stats: accepted=30 dropped=0 bad_signatures=0 bytes=70333 fps=29.7 avg_payload=2344
2026/05/23 14:16:16 consumer stats: accepted=31 dropped=0 bad_signatures=0 bytes=67099 fps=30.3 avg_payload=2164
2026/05/23 14:16:17 consumer stats: accepted=30 dropped=0 bad_signatures=0 bytes=77999 fps=29.8 avg_payload=2599
2026/05/23 14:16:18 consumer stats: accepted=31 dropped=0 bad_signatures=0 bytes=104390 fps=30.3 avg_payload=3367
2026/05/23 14:16:19 consumer stats: accepted=31 dropped=0 bad_signatures=0 bytes=105056 fps=30.3 avg_payload=3388
2026/05/23 14:16:20 consumer stats: accepted=30 dropped=0 bad_signatures=0 bytes=101312 fps=29.8 avg_payload=3377
2026/05/23 14:16:21 consumer stats: accepted=31 dropped=0 bad_signatures=0 bytes=114932 fps=30.3 avg_payload=3707
2026/05/23 14:16:22 consumer stats: accepted=30 dropped=0 bad_signatures=0 bytes=80503 fps=29.8 avg_payload=2683

```

Рис.4.7. Получение и проверка кадров на стороне консюмера

Консюмер после получения кадра выполняет последовательную проверку. Сначала проверяется принадлежность кадра активной сессии по идентификатору сессии. Затем анализируется порядковый номер кадра, что позволяет обнаруживать повторную передачу старых кадров. После этого проверяется временная метка, исключающая использование просроченных данных. На последнем этапе выполняется проверка цифровой подписи Ed25519. Если подпись некорректна, кадр считается недоверенным и отбрасывается.

Демонстрация передачи кадров подтверждает, что разработанное приложение реализует непрерывную проверку подлинности медиапотока. Каждый кадр проверяется отдельно, поэтому повреждение, подмена или повтор одного кадра не нарушают криптографический контекст последующих кадров. Это особенно важно для потоковой передачи в реальном времени, где отдельные устаревшие или некорректные кадры должны отбрасываться без остановки всей сессии.

4.4. Методика и сценарии модульного тестирования

Тестирование разработанного решения выполнялось для проверки корректности работы протокола как в штатных, так и в ошибочных сценариях. Основной акцент был сделан на модульном тестировании, поскольку разработанный протокол состоит из нескольких логически независимых компонентов: процедуры рукопожатия, криптографической проверки, обработки полезной нагрузки, контроля принадлежности кадра к сессии и политики реакции на нарушения целостности медиапотока.

Такой подход позволяет проверять не только общий успешный сценарий работы системы, но и поведение протокола при некорректных входных данных, повреждении подписей, повторной передаче кадров, нарушении порядка следования payload и подмене параметров доверенной сессии.

Для запуска проверок использовался стандартный механизм тестирования языка Go [0]. Результаты выполнения фиксировались в терминале, после чего к работе прикладывались скриншоты с выводом успешного прохождения тестов. Наличие строк PASS и ok в выводе подтверждает, что отдельные тестовые функции и весь проверяемый пакет завершили выполнение без ошибок.

4.4.1. Тесты установления доверенной сессии

В данной группе тестов проверялась корректность процедуры рукопожатия и устойчивость начального этапа установления доверия к некорректным данным. Рукопожатие является критическим этапом протокола, поскольку именно на нём временный публичный ключ Ed25519 связывается с долговременной идентичностью камеры, подтверждённой сертификатом и подписью RSA-PSS.

Были протестированы ситуации с неверной версией протокола, пустым одноразовым случайным значением, лишними байтами в ответе, несовпадением consumer nonce, повреждённой подписью рукопожатия и просроченной временной меткой. Во всех указанных случаях ожидаемым поведением является отклонение запроса, ответа или всей процедуры рукопожатия. Перечень соответствующих тестовых сценариев представлен в таблице 4.1.

Таблица 4.1

Тестовые сценарии установления доверенной сессии

Группа	Событие	Ожидаемый результат	Тест
Доверенная сессия	Корректное рукопожатие	Сессия успешно создаётся и проверяется	TestHandshakeExchangeRoundTrip
Доверенная сессия	Несовпадение consumer nonce	Рукопожатие отклоняется	TestHandshakeExchangeAnomalies/rejects_nonce_mismatch
Доверенная сессия	Повреждённая подпись рукопожатия	Рукопожатие отклоняется	TestHandshakeExchangeAnomalies/rejects_tampered_handshake_signature
Доверенная сессия	Просроченная временная метка	Рукопожатие отклоняется	TestHandshakeExchangeAnomalies/rejects_expired_timestamp
Доверенная сессия	Корректный запрос	Запрос кодируется и декодируется	TestHandshakeMessageRoundTrip/request
Доверенная сессия	Корректный ответ	Ответ кодируется и декодируется	TestHandshakeMessageRoundTrip/response
Доверенная сессия	Неверная версия протокола	Запрос отклоняется	TestHandshakeMessageAnomalies/rejects_invalid_protocol_version
Доверенная сессия	Пустой consumer nonce	Запрос отклоняется	TestHandshakeMessageAnomalies/rejects_empty_consumer_nonce
Доверенная сессия	Лишние байты в ответе	Ответ отклоняется	TestHandshakeMessageAnomalies/rejects_trailing_bytes_in_response

```

=== RUN    TestHandshakeExchangeRoundTrip
--- PASS: TestHandshakeExchangeRoundTrip (0.49s)
=== RUN    TestHandshakeExchangeAnomalies
=== RUN    TestHandshakeExchangeAnomalies/rejects_nonce_mismatch
=== RUN    TestHandshakeExchangeAnomalies/rejects_tampered_handshake_signature
=== RUN    TestHandshakeExchangeAnomalies/rejects_expired_timestamp
--- PASS: TestHandshakeExchangeAnomalies (4.00s)
    --- PASS: TestHandshakeExchangeAnomalies/rejects_nonce_mismatch (2.96s)
    --- PASS: TestHandshakeExchangeAnomalies/rejects_tampered_handshake_signature (0.20s)
    --- PASS: TestHandshakeExchangeAnomalies/rejects_expired_timestamp (0.84s)
=== RUN    TestHandshakeMessageRoundTrip
=== RUN    TestHandshakeMessageRoundTrip/request
=== RUN    TestHandshakeMessageRoundTrip/response
--- PASS: TestHandshakeMessageRoundTrip (0.00s)
    --- PASS: TestHandshakeMessageRoundTrip/request (0.00s)
    --- PASS: TestHandshakeMessageRoundTrip/response (0.00s)
=== RUN    TestHandshakeMessageAnomalies
=== RUN    TestHandshakeMessageAnomalies/rejects_invalid_protocol_version
=== RUN    TestHandshakeMessageAnomalies/rejects_empty_consumer_nonce
=== RUN    TestHandshakeMessageAnomalies/rejects_trailing_bytes_in_response
--- PASS: TestHandshakeMessageAnomalies (0.00s)
    --- PASS: TestHandshakeMessageAnomalies/rejects_invalid_protocol_version (0.00s)
    --- PASS: TestHandshakeMessageAnomalies/rejects_empty_consumer_nonce (0.00s)
    --- PASS: TestHandshakeMessageAnomalies/rejects_trailing_bytes_in_response (0.00s)
PASS

```

Рис.4.8. Результат выполнения модульных тестов установления доверенной сессии

4.4.2. Криптографические тесты целостности данных

В рамках криптографических тестов проверялись операции, связанные с формированием и проверкой цифровых подписей, а также с выпуском сертификата камеры. Основная задача данной группы тестов заключается в подтверждении того, что изменение подписанных данных приводит к ошибке проверки, а криптографические параметры соответствуют заданной политике протокола.

Отдельно проверялась невозможность успешной проверки подписи кадра после изменения payload уже после формирования подписи. Такой сценарий моделирует попытку подмены содержимого медиаданных при сохранении старой подписи. Также проверялась невозможность выпуска сертификата камеры при неверном размере ключа RSA. Перечень проверок криптографического блока представлен в таблице 4.2.

Таблица 4.2

Криптографические тесты целостности данных

Группа	Событие	Ожидаемый результат	Тест
Криптография	Корректная подпись кадра	Подпись проходит проверку	TestFrameSignature/accepts_valid_signature
Криптография	Подмена payload после подписи	Подпись кадра не проходит	TestFrameSignature/rejects_tampered_payload
Криптография	Корректный сертификат камеры	Сертификат выпускается и проверяется	TestCameraCertificate/issue_s_and_verifies_camera_certificate
Криптография	Неверный размер ключа RSA	Сертификат камеры не выпускается	TestCameraCertificate/rejects_wrong_rsa_key_size

```

=== RUN    TestFrameSignature
=== RUN    TestFrameSignature/accepts_valid_signature
=== RUN    TestFrameSignature/rejects_tampered_payload
--- PASS: TestFrameSignature (0.00s)
    --- PASS: TestFrameSignature/accepts_valid_signature (0.00s)
    --- PASS: TestFrameSignature/rejects_tampered_payload (0.00s)
=== RUN    TestCameraCertificate
=== RUN    TestCameraCertificate/issues_and_verifies_camera_certificate
=== RUN    TestCameraCertificate/rejects_wrong_rsa_key_size
--- PASS: TestCameraCertificate (1.76s)
    --- PASS: TestCameraCertificate/issues_and_verifies_camera_certificate (1.00s)
    --- PASS: TestCameraCertificate/rejects_wrong_rsa_key_size (0.76s)
PASS
ok      crypto-stream-auth/internal/crypto    (cached)

```

Рис.4.9. Результат выполнения криптографических модульных тестов

4.4.3. Сетево-протокольные тесты актуальности и порядка передачи

В данной группе тестов проверялась логика обработки кадров внутри активной доверенной сессии. Эти тесты относятся не только к криптографической проверке, но и к сетевому-протокольной логике, поскольку моделируют повторную передачу, нарушение порядка следования кадров и обработку устаревших данных.

Были проверены повторная передача кадра с тем же порядковым номером, получение кадра с меньшим Sequence ID после уже принятого нового кадра и обработка просроченной временной метки. Во всех указанных случаях кадр должен быть отброшен и не должен передаваться дальше в медиаконвейер. Перечень сетевых-протокольных проверок представлен в таблице 4.3.

Таблица 4.3

Сетево-протокольные тесты актуальности и порядка передачи

Группа	Событие	Ожидаемый результат	Тест
Сеть и порядок	Корректный кадр	Кадр принимается	TestFrameValidationRoundTrip
Сеть и порядок	Повторный Sequence ID	Кадр отбрасывается как повторный	TestFrameValidationAnomalies/rejects_replay_sequence
Сеть и порядок	Старый Sequence ID	Кадр отбрасывается как устаревший	TestFrameThreats/threat_07_rejects_old_frame_sequence
Сеть и порядок	Просроченная временная метка	Кадр отбрасывается	TestFrameValidationAnomalies/rejects_expired_timestamp

```

=== RUN    TestFrameSignature
=== RUN    TestFrameSignature/accepts_valid_signature
=== RUN    TestFrameSignature/rejects_tampered_payload
--- PASS: TestFrameSignature (0.00s)
    --- PASS: TestFrameSignature/accepts_valid_signature (0.00s)
    --- PASS: TestFrameSignature/rejects_tampered_payload (0.00s)
=== RUN    TestCameraCertificate
=== RUN    TestCameraCertificate/issues_and_verifies_camera_certificate
=== RUN    TestCameraCertificate/rejects_wrong_rsa_key_size
--- PASS: TestCameraCertificate (1.76s)
    --- PASS: TestCameraCertificate/issues_and_verifies_camera_certificate (1.00s)
    --- PASS: TestCameraCertificate/rejects_wrong_rsa_key_size (0.76s)
PASS
ok      crypto-stream-auth/internal/crypto    (cached)

```

Рис.4.10. Результат выполнения криптографических модульных тестов

4.4.4. Сессионные тесты принадлежности данных

Сессионные тесты проверяют, что консюмер принимает только те кадры, которые относятся к текущей активной медиасессии. Такая проверка необходима для защиты от смешивания данных разных сессий и от попытки передать корректно подписанный, но относящийся к другой сессии payload.

В ходе тестирования кадр формировался с идентификатором сессии, отличающимся от идентификатора активной сессии валидатора. Ожидаемым результатом является отклонение такого кадра до передачи его в дальнейшую обработку. Перечень сессионных проверок представлен в таблице 4.4.

Таблица 4.4

Сессионные тесты принадлежности данных

Группа	Событие	Ожидаемый результат	Тест
Сессия	Чужой Session ID	Кадр отбрасывается	TestFrameValidationAnomalies/rejects_wrong_session_id
Сессия	Payload из другой сессии	Кадр отбрасывается	TestFrameThreats/threat_08_rejects_foreign_session_id

4.4.5. Тесты пороговой политики реакции на нарушения

Отдельно проверялась политика реакции на поток некорректно подписанных кадров. Данный механизм необходим для разграничения единичных ошибок и устойчивого нарушения криптографической целостности медиапотока. Если в коротком временном окне получено слишком мало кадров, ситуация не должна приводить к повторному рукопожатию, поскольку она может быть следствием деградации сети. Если же доля кадров с некорректной подписью превышает заданный порог при достаточном объёме выборки, система должна инициировать повторное рукопожатие, а при повторном нарушении сбросить соединение.

Перечень тестов пороговой политики представлен в таблице 4.5.

Таблица 4.5

Тесты пороговой политики реакции на нарушения

Группа	Событие	Ожидаемый результат	Тест
Политика реакции	Меньше или равно 30 кадров в окне	Повторное рукопожатие не запускается	TestFramePolicyAnomalies/ignores_small_sample
Политика реакции	Более 20% кадров с некорректной подписью	Сначала запускается повторное рукопожатие, затем выполняется сброс соединения	TestFramePolicyAnomalies/requests_rehandshake_then_drops_connection


```

=== RUN    TestFramePolicyAnomalies
=== RUN    TestFramePolicyAnomalies/ignores_small_sample
=== RUN    TestFramePolicyAnomalies/requests_rehandshake_then_drops_connection
--- PASS: TestFramePolicyAnomalies (0.00s)
    --- PASS: TestFramePolicyAnomalies/ignores_small_sample (0.00s)
    --- PASS: TestFramePolicyAnomalies/requests_rehandshake_then_drops_connection (0.00s)
=== RUN    TestFrameValidationRoundTrip
--- PASS: TestFrameValidationRoundTrip (0.00s)
=== RUN    TestFrameValidationAnomalies
=== RUN    TestFrameValidationAnomalies/rejects_replay_sequence
=== RUN    TestFrameValidationAnomalies/rejects_wrong_session_id
=== RUN    TestFrameValidationAnomalies/rejects_expired_timestamp
=== RUN    TestFrameValidationAnomalies/rejects_bad_signature
--- PASS: TestFrameValidationAnomalies (0.00s)
    --- PASS: TestFrameValidationAnomalies/rejects_replay_sequence (0.00s)
    --- PASS: TestFrameValidationAnomalies/rejects_wrong_session_id (0.00s)
    --- PASS: TestFrameValidationAnomalies/rejects_expired_timestamp (0.00s)
    --- PASS: TestFrameValidationAnomalies/rejects_bad_signature (0.00s)
PASS
ok      crypto-stream-auth/internal/stream    (cached)

```

Рис.4.11. Результат выполнения модульных тестов потоковой обработки и пороговой политики

4.5. Выводы по результатам тестирования

Проведённое модульное тестирование подтвердило корректность работы основных компонентов разработанного протокола. Тесты установления доверенной сессии показали, что рукопожатие отклоняется при несовпадении одноразового значения, повреждении подписи, просроченной временной метке и нарушении формата сообщений. Это подтверждает защищённость начального этапа установления доверия.

Криптографические тесты подтвердили, что изменение payload после формирования подписи приводит к ошибке проверки Ed25519-подписи. Следовательно, консюмер способен обнаруживать подмену медиаданных и отбрасывать недоверенные кадры.

Сетево-протокольные тесты показали, что повторные, устаревшие и просроченные кадры не принимаются системой. Это подтверждает работоспособность механизма контроля Sequence ID и временных меток, необходимых для защиты от replay-атаки и нарушения порядка передачи.

Сессионные тесты подтвердили, что кадры с чужим Session ID отклоняются, даже если они имеют корректную структуру. Это позволяет предотвратить смешивание данных разных медиасессий.

Тесты пороговой политики показали, что единичные ошибки подписи не приводят к немедленному разрыву соединения, однако устойчивое превышение доли некорректных кадров инициирует повторное рукопожатие и последующий сброс соединения. Таким образом, реализованная политика позволяет отличать сетевую

деградацию от систематического нарушения криптографической целостности медиапотока.

ГЛАВА 5. АНАЛИЗ РЕЗУЛЬТАТОВ

В ходе разработки была получена программная реализация протокола непрерывной криптографической аутентификации медиапотока в реальном времени. Основной результат заключается в том, что проверка доверия выполняется на стороне получателя, а сервер сохраняет роль маршрутизатора и не становится доверенной стороной.

5.1. Вычислительная нагрузка

Вычислительная нагрузка распределена следующим образом:

- продюсер подписывает каждый кадр временным ключом Ed25519;
- консюмер проверяет подпись каждого кадра;
- сервер не проверяет подписи кадров и не выполняет криптографическую обработку медиаданных.

Такое разделение снижает нагрузку на сервер и позволяет масштабировать маршрутизацию потока. Серверу не нужно выполнять дорогостоящую проверку каждого кадра для каждого консюмера.

При этом основная нагрузка переносится на консюмера. Это допустимо, так как именно консюмер принимает решение о доверии к потоку. Если кадр поврежден, подменен, повторен или относится к другой сессии, он отбрасывается до использования в дальнейшем медиаконвейере.

Отдельно следует учитывать стоимость копирования с полезной нагрузки. В текущей модели подписываемое сообщение кадра включает идентификатор сессии, порядковый номер кадра, временную метку и полезную нагрузку, поэтому при подписи и проверке создается буфер с данными для подписи. Для больших кадров это может давать дополнительную нагрузку на память. В дальнейшем эту часть можно оптимизировать, уменьшив количество промежуточных копирований.

5.2. Сетевые накладные расходы

Каждый кадр передается вместе с метаданными, необходимыми для проверки подлинности:

- Session ID (идентификатор сессии) — 16 байт;
- Sequence ID (порядковый номер кадра) — 8 байт;
- Timestamp (временная метка) — 8 байт;
- длина Payload (полезной нагрузки) — 4 байта;
- подпись Ed25519 — 64 байта [0].

Итого постоянные накладные расходы на один кадр составляют 100 байт без учета служебных данных QUIC.

Для видеопотока это приемлемо, так как размер одного видеокadra обычно значительно больше 100 байт. При этом добавляемые поля дают важные свойства безопасности: защиту от подмены, повторной передачи, смешивания сессий и использования устаревших кадров.

5.3. Применимость решения

Разработанное решение применимо для непрерывной криптографической аутентификации медиапотока в реальном времени, так как тяжелая операция с RSA и TPM выполняется только на этапе установления доверенной сессии, а постоянная проверка кадров выполняется с помощью Ed25519.

Выбранная архитектура сохраняет сквозное доверие: даже если сервер передает поток, он не может незаметно изменить кадр, так как консюмер самостоятельно проверяет подпись каждого кадра. Это делает решение пригодным для сценариев, где важно подтвердить происхождение видеоданных от доверенной камеры и обнаружить изменение потока при передаче через сеть.

Основными ограничениями текущей реализации являются необходимость дальнейших нагрузочных измерений, проверка поведения при большом числе одновременных консюмеров и оптимизация копирования полезной нагрузки. Тем не менее функциональные тесты подтверждают корректность ключевых защитных механизмов протокола.

5.4. Анализ результатов

В результате выполнения экспериментальной проверки была подтверждена работоспособность разработанного протокола непрерывной криптографической аутентификации медиапотока в реальном времени. Система обеспечила передачу медиаданных от продюсера к консюмеру через промежуточный сервер с сохранением возможности независимой проверки подлинности потока на стороне получателя.

В нормальном сценарии работы продюсер успешно формировал медиасессию, выполнял handshake с консюмером и передавал payload, содержащий фрагменты MPEG-TS с H.264-видео. Консюмер корректно проверял сертификат камеры, подпись handshake-контекста и подписи полезной нагрузки с использованием временного публичного ключа Ed25519. Это подтверждает, что сервер в предложенной архитектуре не является доверенным участником с точки зрения криптографической проверки, а выполняет роль маршрутизатора медиаданных.

Отдельная проверка показала, что при использовании недоверенного или некорректного сертификата камера не принимается системой как доверенный источник. Следовательно, механизм TrustStore, основанный на доверенных Root CA и манифесте камер, обеспечивает связь между конкретной камерой, её сертификатом и допустимым корневым центром сертификации. Это снижает риск подключения произвольного источника, не зарегистрированного в системе.

Проверка handshake показала, что изменение consumer nonce, Session ID, временной метки или временного публичного ключа приводит к отклонению handshake-ответа. Это подтверждает, что подпись handshake-контекста защищает начальный этап установления доверия от подмены параметров сессии и повторного использования ранее перехваченных данных.

При проверке передачи payload было установлено, что изменение полезной нагрузки, порядкового номера, идентификатора сессии или временной метки приводит к ошибке проверки подписи. Таким образом, подпись payload обеспечивает контроль целостности не только самих медиаданных, но и служебных параметров, необходимых для защиты от повторов, перестановки и подмены фрагментов потока.

Также была подтверждена корректность работы механизма обработки последовательности. Консюмер отбрасывал повторные payload и payload с некорректным

Sequence ID. Это позволяет снизить риск replay-атаки, при которой злоумышленник пытается повторно передать ранее перехваченный фрагмент медиапотока.

В ходе проверки пороговой политики было установлено, что одиночные ошибки подписи не приводят к немедленному разрыву соединения. При этом превышение доли некорректно подписанных payload более 20% за трёхсекундное окно при количестве полученных payload более 30 инициирует повторный handshake. Такой подход позволяет отделить случайные сбои и деградацию сети от устойчивого нарушения криптографической целостности потока.

Переход от обработки отдельных NALU к универсальной модели подписываемого payload показал, что криптографический уровень не зависит от внутренней структуры видеокодека. В текущей реализации внутри payload передаются данные MPEG-TS, содержащие H.264-видео, однако сам протокол проверяет только ограниченную по размеру полезную нагрузку и связанные с ней метаданные. Это повышает универсальность решения и упрощает дальнейшую поддержку других кодеков и контейнеров.

Дополнительно была проверена работа системы с несколькими консьюмерами. Сервер использовал общий media hub для рассылки одного входящего медиапотока нескольким получателям. Для каждого нового консьюмера выполнялся отдельный handshake с уникальным consumer nonce, при этом активная Session ID и временный ключ Ed25519 сохранялись в рамках текущей медиасессии. Это подтверждает возможность масштабирования схемы на несколько получателей без необходимости создавать отдельный медиапоток для каждого консьюмера.

Таким образом, результаты тестирования подтверждают, что разработанная система обеспечивает проверку доверенного источника, защиту handshake от подмены, контроль целостности payload, обнаружение повторов и реакцию на устойчивые нарушения криптографической целостности. Основным результатом работы заключается в программной реализации протокола, при котором сервер не обязан проверять каждый фрагмент медиапотока, а конечный получатель самостоятельно устанавливает доверие к источнику и проверяет подлинность принимаемых данных.

ЗАКЛЮЧЕНИЕ

В ходе работы был рассмотрен подход к криптографической аутентификации медиапотока в реальном времени. Были проанализированы особенности потоковой передачи видеоданных, угрозы подмены, повторного воспроизведения и внедрения синтетического контента.

Поставленные задачи решены: исследованы криптографические методы защиты медиапотока, определены ограничения существующих решений, сформулированы требования к протоколу и предложена структура криптографической метки. Метка связывает данные сессии, номер кадра, временную метку и хеш полезной нагрузки, что позволяет проверять целостность и актуальность видеокадров.

Также была предложена архитектура решения с продюсером, сервером-маршрутизатором и консюмером. В ней сервер выполняет маршрутизацию, а окончательная проверка подлинности кадра выполняется на стороне получателя.

Таким образом, цель работы достигнута. Разработанный подход позволяет обнаруживать модификацию, повтор, перестановку и устаревание видеокадров и может применяться в системах защищенных видеотрансляций, удаленной идентификации и телемедицины.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

0. A Guide to Replay Attacks And How to Defend Against Them. — URL: <https://www.packetlabs.net/posts/a-guide-to-replay-attacks-and-how-to-defend-against-them/> (дата обращения: 22.04.2026).
0. Anti-replay. — URL: <https://en.wikipedia.org/wiki/Anti-replay> (дата обращения: 16.04.2026).
0. Benchmarking wolfSSL and wolfCrypt. — URL: <https://www.wolfssl.com/docs/benchmarks/> (дата обращения: 25.04.2026).
0. C2PA Implementation Guidance. — URL: <https://spec.c2pa.org/specifications/specifications/2.3/guidance/Guidance.html> (дата обращения: 27.04.2026).
0. Comparing SSH Keys: RSA, DSA, ECDSA, or EdDSA? — URL: <https://goteleport.com/blog/comparing-ssh-keys/> (дата обращения: 26.04.2026).
0. Content Credentials: C2PA Technical Specification. — URL: https://spec.c2pa.org/specifications/specifications/2.1/specs/_attachments/C2PA_Specification.pdf (дата обращения: 24.04.2026).
0. Content Credentials: C2PA Technical Specification. — URL: https://spec.c2pa.org/specifications/specifications/2.4/specs/C2PA_Specification.html (дата обращения: 26.04.2026).
0. Cryptographic Accelerators for Digital Signature Based on Ed25519. — URL: <https://cse.usf.edu/~mehran2/Papers/J46.pdf> (дата обращения: 24.04.2026).
0. Deepfake Presentation and Injection Attacks: Risks in ID Authentication. — URL: <https://risk.lexisnexis.com/global/en/insights-resources/article/deepfake-duo-presentation-injection-attacks> (дата обращения: 24.04.2026).
0. Difference between SRT and QUIC. — URL: https://www.reddit.com/r/VIDEOENGINEERING/comments/1hlwgrp/difference_between_srt_and_quic/ (дата обращения: 18.04.2026).
0. Digital Signatures: Mechanics and Go Benchmarks. — URL: <https://dev.to/kanywst/digital-signatures-mechanics-and-go-benchmarks-rsa-vs-ecdsa-vs-ed25519-2d36> (дата обращения: 26.04.2026).
0. draft-sharabayko-srt-over-quic-00. — URL: <https://datatracker.ietf.org/doc/draft-sharabayko-srt-over-quic/00/> (дата обращения: 27.04.2026).
0. ECDSA vs ECDH vs Ed25519 vs Curve25519. — URL: <https://security.stackexchange.com/questions/50878/ecdsa-vs-ecdh-vs-ed25519-vs-curve25519> (дата обращения: 25.04.2026).

0. Ed25519. — URL: <https://ed25519.cr.yp.to/> (дата обращения: 15.04.2026).

0. ED25519 Signature: What Is It and How To Use It for Binance API Security. — URL: <https://academy.binance.com/hr-HR/articles/ed25519-signature-what-is-it-and-how-to-use-it-for-binance-api-security> (дата обращения: 20.04.2026).

0. Hardware-backed device attestation powers mobile workers. — URL: <https://techcommunity.microsoft.com/blog/microsoftintuneblog/hardware-backed-device-attestation-powers-mobile-workers/3881209> (дата обращения: 24.04.2026).

0. How C2PA Content Credentials Work and What Their Limits Are. — URL: <https://www.softwareseni.com/how-c2pa-content-credentials-work-and-what-their-limits-are/> (дата обращения: 20.04.2026).

0. Injection Attack Detection for Digital Identity Verification. — URL: <https://www.miteksystems.com/solutions/injection-attack-detection> (дата обращения: 23.04.2026).

0. Injection attack: a major threat against remote identity verification. — URL: <https://pages.nist.gov/ifpc/2025/presentations/09.pdf> (дата обращения: 21.04.2026).

0. Live Streaming Using SRT with QUIC Datagrams. — URL: <https://medium.com/innovation-labs-blog/live-streaming-using-srt-with-quic-datagrams-7896f7ce7bf3> (дата обращения: 27.04.2026).

0. Media Over QUIC and the Future of High-Quality, Low-Latency Streaming. — URL: <https://www.streamingmedia.com/Articles/Editorial/Featured-Articles/Media-Over-QUIC-and-the-Future-of-High-Quality-Low-Latency-Streaming-167165.aspx> (дата обращения: 20.04.2026).

0. NIST Special Publication 1800-34B: Validating the Integrity of Computing Devices. — URL: <https://www.nccoe.nist.gov/sites/default/files/2022-12/tpm-sca-nist-sp1800-34b-final.pdf> (дата обращения: 20.04.2026).

0. Open Measurement Device Attestation Implementation Guidance. — URL: <https://iabtechlab.com/wp-content/uploads/2025/10/Open-Measurement-Device-Attestation-Implementation-Guidance.pdf> (дата обращения: 15.04.2026).

0. Overpromising on Digital Provenance and Security. — URL: <https://www.rand.org/pubs/commentary/2025/06/overpromising-on-digital-provenance-and-security.html> (дата обращения: 22.04.2026).

0. Package testing. — URL: <https://pkg.go.dev/testing> (дата обращения: 16.05.2026).

0. Physical TPM Attestation Keys and certificates. — URL: <https://docs.trustauthority.intel.com/main/articles/articles/ita/tpm-ak-provision.html> (дата обращения: 24.04.2026).

0. Portable C implementation of Ed25519, a high-speed high-security public-key signature system. — URL: <https://github.com/orlp/ed25519> (дата обращения: 20.04.2026).

0. RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA). — URL: <https://www.rfc-editor.org/rfc/rfc8032.html> (дата обращения: 18.05.2026).

0. RSA vs ECDSA vs Ed25519: Choosing the Right Key Type. — URL: <https://getacert.com/learn/key-types-explained> (дата обращения: 26.04.2026).

0. SHA-256 vs SHA-512, Key Differences, Performance and When to Use Each. — URL: <https://qodex.ai/blog/sha256-vs-sha512> (дата обращения: 25.04.2026).

0. SHA-256 vs SHA-512: Compare Top Cryptographic Hashing Algorithms. — URL: <https://mojoauth.com/compare-hashing-algorithms/sha-256-vs-sha-512> (дата обращения: 26.04.2026).

0. SHA-512 faster than SHA-256? — URL: <https://crypto.stackexchange.com/questions/26336/sha-512-faster-than-sha-256> (дата обращения: 24.04.2026).

0. Signing Right Away. — URL: <https://arxiv.org/html/2510.09656v1> (дата обращения: 22.04.2026).

0. Size, Speed, and Security: An Ed25519 Case Study. — URL: https://www.researchgate.net/publication/356187556_Size_Speed_and_Security_An_Ed25519_Case_Study (дата обращения: 24.04.2026).

0. TPM Key Attestation. — URL: <https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/manage/component-updates/tpm-key-attestation> (дата обращения: 28.04.2026).

0. Trusted Capture Device Attestation. — URL: <https://www.envisioning.com/research/beacon/trusted-capture-attestation> (дата обращения: 21.04.2026).

0. Trusted Platform Module fundamentals. — URL: <https://learn.microsoft.com/en-us/windows/security/hardware-security/tpm/tpm-fundamentals> (дата обращения: 16.04.2026).

0. Virtual camera detection: Catching video injection attacks in remote biometric systems. — URL: <https://arxiv.org/html/2512.10653v1> (дата обращения: 24.04.2026).

0. What are injection attacks, and how to prevent them. — URL: <https://veridas.com/en/injection-attacks/> (дата обращения: 21.04.2026).